SRS

FACTS Case Management System

Software Requirements Specification

IEEE 830-1998 (Adapted)

VERSION	DATE	STATUS
3.7	April 23, 2025	Final

• LIVE & OPERATIONAL

Software Requirements Specification (SRS)

FACTS — File and Case Tracking System

Document Version: 3.7
Date: April 23, 2025
Prepared By: FACTS Development Team
Document Status: Final
Classification: Internal
Standard: IEEE 830-1998 (Adapted)

Document Control

ITEM	DETAIL
Project Name	FACTS — File and Case Tracking System
Document Type	Software Requirements Specification (SRS)
Version	3.7
Date	April 23, 2025
Organizations	Baaseteen & SHND (Social Help and Development)
Backend Port	5000 (proxied by Nginx on port 80/443)
Database	baaseteen_cms (MySQL 8.0)
Default Credentials	admin / password (must be changed on first login)

Table of Contents

1. Introduction
2. 1.1 Purpose
3. 1.2 Scope
4. 1.3 Definitions, Acronyms, and Abbreviations
5. 1.4 References
6. 1.5 Overview
7. Overall Description
8. 2.1 Product Perspective
9. 2.2 Product Functions
10. 2.3 User Classes and Characteristics
11. 2.4 Operating Environment
12. 2.5 Design and Implementation Constraints
13. 2.6 Assumptions and Dependencies
14. System Features and Functional Requirements
15. 3.1 Authentication and Session Management
16. 3.2 User Management
17. 3.3 Case Identification and Eligibility Screening
18. 3.4 Case Management and Lifecycle Tracking
19. 3.5 Counseling Forms — Digital Assessment (8 Sections)
20. 3.6 Multi-Level Workflow and Approvals
21. 3.7 Cover Letter Generation
22. 3.8 Payment and Financial Management
23. 3.9 Fund Utilization and Quarterly Reporting
24. 3.10 Welfare Checklist
25. 3.11 Document Management and Attachments
26. 3.12 Notification System
27. 3.13 Dashboard Analytics and Reporting
28. 3.14 Master Data Management
29. External Interface Requirements
30. 4.1 User Interfaces
31. 4.2 Hardware Interfaces
32. 4.3 Software Interfaces
33. 4.4 Communication Interfaces
34. Non-Functional Requirements
35. 5.1 Performance Requirements
36. 5.2 Safety Requirements
37. 5.3 Security Requirements
38. 5.4 Software Quality Attributes
39. 5.5 Scalability Requirements
40. Database Specification
41. 6.1 Database Technology
42. 6.2 Schema Overview
43. 6.3 Table Definitions
44. 6.4 Relationships and Constraints
45. 6.5 Data Dictionary
46. System Architecture
47. 7.1 Architectural Pattern
48. 7.2 Component Diagram
49. 7.3 Request Lifecycle
50. 7.4 Directory Structure
51. Data Flow Specifications

52. 8.1 Context Diagram (Level 0)
53. 8.2 Level 1 DFD
54. 8.3 Process Specifications
55. Deployment Specification
56. 9.1 Production Environment
57. 9.2 Local Development Environment
58. 9.3 Process Management and Reverse Proxy
59. Appendices
    - A: Use Case Catalog
    - B: Error Handling Specification
    - C: File Upload Specifications
    - D: API Endpoint Inventory
    - E: Configuration Parameters

---

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a complete and detailed description of the requirements for the **FACTS — File and Case Tracking System**. It is intended for:

- **Developers** — to understand technical requirements and implement features correctly
- **Testers** — to create test cases and validate system behavior against acceptance criteria
- **Project Managers** — to track scope, prioritize backlog, and monitor delivery progress
- **Stakeholders** — to understand system capabilities and approve functional specifications
- **Maintenance Team** — to reference current specifications during updates and troubleshooting

This document describes the software's functional requirements, external interfaces, design constraints, performance attributes, and all non-functional requirements, organized per the IEEE 830-1998 standard.

1.2 Scope

FACTS is a comprehensive, full-stack web application that digitalizes and centralizes the complete lifecycle of welfare case management for the Baaseteen and SHND organizations. The system manages:

1. **User account and role management** — JWT authentication, granular RBAC, Jamiat/Jamaat scope isolation, multi-role user support
2. **Case identification and eligibility screening** — pre-assessment intake with ITS API integration, income assessment, and duplicate detection
3. **Case lifecycle management** — creation, assignment, status tracking, SLA monitoring, audit trail, and closure
4. **Digital counseling forms** — 8-section wizard covering Personal Details, Family Details, Assessment, Financial Assistance, Economic Growth, Declaration, Attachments, and Manzoori
5. **Multi-level workflow approvals** — configurable per case type, supporting up to 10 stages and 5 executive approval tiers, with approve/reject/rework flows
6. **Cover letter generation** — automated PDF generation from consolidated case data, with welfare department digital signature capture
7. **Payment and financial management** — Qardan Hasana and Enayat disbursement schedules, installment tracking, and repayment confirmation
8. **Fund utilization reporting** — quarterly reassessment forms with configurable admin-defined question schema, photo uploads, and PDF export
9. **Welfare checklist** — configurable checklist with drag-and-drop categories and per-case completion enforcement
10. **Document management** — multi-file attachment support per case and counseling section, with type and size validation
11. **Notification system** — in-app notifications for case assignments, status changes, and SLA alerts
12. **Dashboard analytics** — role-scoped statistics, pipeline overviews, and recent activity feeds
13. **Reporting** — Excel import/export for applicants and users; PDF downloads for cover letters and quarterly reports
14. **Master data management** — configuration of Jamiat, Jamaat, case types, workflow stages, executive levels, relations, education levels, and occupations

Out of Scope: Mobile native apps, SMS notifications, video conferencing, payment gateway integration, public applicant self-registration portal, multi-tenant SaaS architecture, third-party CRM integration.

1.3 Definitions, Acronyms, and Abbreviations

TERM	DEFINITION
FACTS	File and Case Tracking System — the system described in this document
ITS	Institutional Information System — external API service for applicant demographic data
JWT	JSON Web Token — stateless authentication token format (RFC 7519)
RBAC	Role-Based Access Control — permission model using resource:action pairs
DCM	Deputy Case Manager — senior case coordination role responsible for counselor assignment
ZI	Zonal Incharge — mid-tier approval authority in the workflow hierarchy

TERM	DEFINITION
KG	Senior approval role between ZI Approval and Operations Lead in the workflow
Jamiat	Top-level organizational geographic unit for user access scoping
Jamaat	Sub-organizational unit within a Jamiat; finest scope level for Counselors
Manzoori	Final sanction/approval stage representing formal case clearance for disbursement
Qardan / Qardan Hasana	Interest-free loan type of financial assistance that requires repayment
Enayat	Grant-based financial assistance that does not require repayment
SLA	Service Level Agreement — time threshold per workflow stage triggering alerts on breach
SPA	Single Page Application — the React frontend architecture
ORM	Object-Relational Mapping — abstraction layer for database queries (mysql2 used directly)
CRUD	Create, Read, Update, Delete — standard data operations
Multer	Node.js middleware for handling multipart/form-data file uploads
Puppeteer	Node.js library for headless Chrome, used for HTML-to-PDF generation
PDFKit	Node.js library for programmatic PDF document generation
PM2	Production process manager for Node.js applications

1.4 References

#	DOCUMENT	FILE
1	Technical Documentation / README	`README (1).md`
2	API Schema Reference	`API_SCHEMAS.md`
3	Database Schema (Mermaid)	`baaseteen_cms.md`
4	Business Requirements Document	`FACTS_BRD_v3.7.html`
5	IEEE Standard 830-1998	Software Requirements Specifications

1.5 Overview

This SRS is organized per the IEEE 830-1998 standard:

- **Section 1** introduces the document and defines terminology
- **Section 2** gives overall system description, user classes, and constraints
- **Section 3** specifies all functional requirements organized by module (14 modules)
- **Section 4** details external interface requirements (UI, hardware, software, communication)
- **Section 5** covers non-functional requirements (performance, security, quality)
- **Section 6** provides the complete database specification with table definitions
- **Section 7** describes the 3-tier system architecture
- **Section 8** presents data flow specifications
- **Section 9** covers deployment specifications (PM2, Nginx, environment)
- **Section 10** contains appendices with supplementary specifications

2. Overall Description

2.1 Product Perspective

FACTS is a **standalone, three-tier web application** that replaces manual paper-based welfare processes with a centralized digital platform. It is not a component of a larger system. The system digitizes and centralizes all stages of welfare case operations previously managed through:

- Paper-based counseling forms and welfare assessment folders
- Spreadsheet-based case tracking and status reporting
- Manual disbursement ledgers and paper payment records
- Ad hoc email-based approval workflows across organizational hierarchies
- Decentralized applicant records with no duplicate detection

System Context Diagram:

```
                    ┌──────────────────────┐
                    │  Web Browsers        │
                    │ (Chrome, Firefox,    │
                    │  Safari, Edge)       │
                    └──────────────────────┘
                              │ HTTPS
                              ▼
┌──────────────┐   ┌──────────────────────┐   ┌──────────────┐
│ Excel/CSV    │──►│                      │   │ Puppeteer    │
│ Imports      │   │  FACTS               │──►│ (PDF Gen)    │
└──────────────┘   │  Case Management     │   └──────────────┘
                   │  System              │
┌──────────────┐   │  Node.js + Express   │   ┌──────────────┐
│ File         │──►│  React 18 (SPA)      │──►│ PDFKit       │
│ Attachments  │   │  (port 5000 → Nginx) │   │ (PDF Gen)    │
└──────────────┘   └──────────────────────┘   └──────────────┘
                              │
                  ┌───────────┼──────────────┐
                  ▼           ▼              ▼
      ┌──────────────────┐ ┌──────────┐ ┌──────────────┐
      │  MySQL Database  │ │ Local    │ │ External     │
      │  baaseteen_cms   │ │ File     │ │ ITS API      │
      │  (50+ tables)    │ │ System   │ │ (Applicant   │
      └──────────────────┘ │ (uploads)│ │ Lookup)      │
                           └──────────┘ └──────────────┘
```

2.2 Product Functions

The system provides the following major function groups:

#	FUNCTION GROUP	DESCRIPTION
F1	Authentication & RBAC	JWT login, role-based permissions, Jamiat/Jamaat scope isolation, multi-role support
F2	User Management	CRUD for all 8 roles, bulk Excel import/export, executive level assignment
F3	Case Identification	Pre-screening intake, ITS auto-fetch, income assessment, eligibility decision, duplicate detection
F4	Case Management	Case creation, assignment, status tracking, SLA monitoring, audit trail, closure
F5	Counseling Forms	8-section digital counseling wizard with partial saves, digital signatures, and file attachments
F6	Workflow & Approvals	Configurable multi-stage pipelines, approve/reject/rework actions, role-gated transitions
F7	Cover Letters	Automated PDF cover letter from consolidated case data, with welfare signature capture
F8	Payment Management	Qardan/Enayat disbursement schedules, installment tracking, finance confirmation

#	FUNCTION GROUP	DESCRIPTION
F9	Fund Utilization	Quarterly reassessment forms, configurable question schema, photo uploads, PDF export
F10	Welfare Checklist	Configurable checklist per case type, per-case completion tracking and enforcement
F11	Attachments	Multi-file upload per case/section, type and size validation, download capability
F12	Notifications	In-app alerts for assignments, status changes, and SLA breaches; unread badge count
F13	Dashboard & Reports	Role-scoped statistics, pipeline analytics, Excel export for bulk data
F14	Master Data	Admin configuration of Jamiat, Jamaat, case types, workflow stages, executive levels, and dropdown dictionaries

2.3 User Classes and Characteristics

2.3.1 Super Admin

ATTRIBUTE	DETAIL
Frequency	Daily
Technical Skill	High
Responsibilities	System governance, all CRUD operations, role configuration, master data management, technical troubleshooting
Estimated Count	1–2
Special Needs	Full access with no Jamiat/Jamaat scope restrictions; can manage roles, permissions, and system-level settings

2.3.2 Admin

ATTRIBUTE	DETAIL
Frequency	Daily
Technical Skill	Medium–High
Responsibilities	User management, master data maintenance, case oversight, report generation, operational configuration
Estimated Count	2–5
Special Needs	Global scope access; cannot modify core system roles or delete master data with active dependencies

2.3.3 DCM (Deputy Case Manager)

ATTRIBUTE	DETAIL
Frequency	Daily
Technical Skill	Medium
Responsibilities	Case coordination, counselor assignment, case intake, workflow management within assigned Jamiat
Estimated Count	5–15
Special Needs	Scoped to assigned Jamiat; can create and reassign cases; efficiency in bulk assignment workflows

2.3.4 Counselor

ATTRIBUTE	DETAIL
Frequency	Multiple times daily
Technical Skill	Low–Medium
Responsibilities	Counseling form completion across all 8 sections, digital signature capture, document attachment, client data entry
Estimated Count	20–50
Special Needs	Scoped to assigned Jamaat; intuitive section-by-section form navigation; partial save without losing progress; mobile-compatible interface

2.3.5 Welfare Reviewer

ATTRIBUTE	DETAIL
Frequency	Daily during active review cycles
Technical Skill	Medium
Responsibilities	Review counseling completions, complete welfare checklist, approve/reject/rework cases at the Welfare Review stage
Estimated Count	5–10
Special Needs	Quick access to counseling form summary view; welfare checklist enforcement before approval; rework notes visible in case history

2.3.6 Executive

ATTRIBUTE	DETAIL
Frequency	Weekly to Monthly
Technical Skill	Low–Medium
Responsibilities	Multi-level financial sanction approvals (up to 5 levels); executive approval queue management; override or forward cases
Estimated Count	5–15
Special Needs	Filtered queue showing only cases at their executive level; approve/reject/forward with remarks; cover letter PDF accessible

2.3.7 Finance

ATTRIBUTE	DETAIL
Frequency	Weekly
Technical Skill	Medium
Responsibilities	Disbursement confirmation on individual payment installments, payment schedule review, financial reconciliation
Estimated Count	3–8
Special Needs	Access to payment schedule lists; one-click disbursement confirmation with timestamp recording

2.3.8 Report Viewer

ATTRIBUTE	DETAIL
Frequency	Weekly to Monthly
Technical Skill	Low
Responsibilities	Dashboard consumption, pipeline analysis, report downloads; read-only access
Estimated Count	5–10
Special Needs	Read-only access to assigned scope; no data modification capability; Excel export for downstream analysis

2.4 Operating Environment

2.4.1 Production Server

COMPONENT	SPECIFICATION
Operating System	Linux (Ubuntu 20.04+ or CentOS 7+)
Runtime	Node.js 16+ (LTS)
Package Manager	npm 8+
Database	MySQL 8.0+ (InnoDB engine)
Process Manager	PM2 (or systemd service)
Reverse Proxy	Nginx (port 80/443 → port 5000)
SSL	Let's Encrypt / Certbot (recommended)
Backend Port	5000 (internal)
Database Name	baaseteen_cms

2.4.2 Local Development Environment

COMPONENT	SPECIFICATION
Platform	Windows / macOS / Linux
Node.js	16+
Database	MySQL 8.0 (local instance or Docker)
Frontend Dev Server	Vite (default port 5173)
Backend Dev Port	5000
API Prefix	`/api`

2.4.3 Client Requirements

REQUIREMENT	SPECIFICATION
Browser	Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
JavaScript	Required (ES6+, React 18)
Screen Resolution	Minimum 360px width (mobile); 1024px+ recommended
Internet	Stable broadband (required for ITS API integration)

2.5 Design and Implementation Constraints

#	CONSTRAINT	DESCRIPTION
DC-1	**Backend Language**	Node.js 16+ with Express.js framework; CommonJS module system
DC-2	**Frontend Framework**	React 18.2 SPA; build tool: Vite; state: Zustand + React Query; forms: React Hook Form
DC-3	**Database**	MySQL 8.0+ with mysql2 driver; no ORM — raw parameterized SQL queries
DC-4	**Authentication**	Stateless JWT (jsonwebtoken library); 24-hour token expiry; bcryptjs for password hashing (10 rounds)
DC-5	**Styling**	Tailwind CSS v3.4 utility classes; no component library dependency
DC-6	**PDF Generation**	Puppeteer for cover letter PDFs (HTML-to-PDF); PDFKit for programmatic report generation
DC-7	**File Handling**	Multer for multipart uploads; local filesystem storage only; no cloud storage integration
DC-8	**Excel Processing**	ExcelJS library for import/export operations
DC-9	**API Design**	RESTful JSON API; all endpoints prefixed with `/api`; standard HTTP status codes
DC-10	**Timezone**	Server timezone per host OS; all timestamps stored in UTC (MySQL TIMESTAMP type)
DC-11	**Character Set**	utf8mb4 (full Unicode support including emoji and Arabic text)
DC-12	**Architecture**	3-tier: React SPA → Express API → MySQL; no microservices; single monolithic backend
DC-13	**Email**	Nodemailer with configurable SMTP relay for notification emails

2.6 Assumptions and Dependencies

Assumptions

ID	ASSUMPTION
A-1	Users have access to modern web browsers with JavaScript enabled
A-2	Reliable internet connectivity is available for ITS API calls and SMTP email
A-3	Concurrent users will not exceed 100 simultaneously under normal operations
A-4	The external ITS API maintains a stable endpoint contract and response schema
A-5	Organizational hierarchy (Jamiat/Jamaat) is stable and configured before user onboarding
A-6	Workflow stages per case type are finalized before cases enter production
A-7	Administrators possess sufficient technical skill for system configuration and user management
A-8	File attachments will not exceed server storage limits; periodic archival is handled operationally
A-9	All users receive adequate role-specific training before accessing the production system
A-10	Legacy data migration is handled separately outside the scope of this specification

Dependencies

ID	DEPENDENCY	IMPACT IF UNAVAILABLE
D-1	Node.js 16+ runtime	Backend application cannot execute
D-2	MySQL 8.0 database accessible	All data operations fail
D-3	External ITS API	Applicant auto-fetch fails; manual entry required

ID	DEPENDENCY	IMPACT IF UNAVAILABLE
D-4	Puppeteer (Chromium)	Cover letter PDF generation unavailable
D-5	SMTP relay server	Email notifications unavailable; in-app notifications unaffected
D-6	Filesystem write access	File uploads and PDF storage fail
D-7	PM2 / Nginx	Production process management and SSL termination unavailable
D-8	npm packages (package.json)	Application fails to start if dependencies are not installed

3. System Features and Functional Requirements

3.1 Authentication and Session Management

3.1.1 Description

The authentication module provides stateless JWT-based login, session management, and role enforcement for all system users. RBAC is enforced at the API route level using a `resource:action` permission model. All data queries are additionally filtered by Jamiat/Jamaat scope at the database layer.

3.1.2 Functional Requirements

SRS-AUTH-001: JWT Login

ATTRIBUTE	DETAIL
Priority	Critical
Access	Public (unauthenticated)
Input	`username` (string, required) or `email` (string, required); `password` (string, min 8 chars, required)
Processing	1. Validate input fields; 2. Lookup user by username or email in `users` table; 3. Check `is_active = 1`; reject inactive accounts with 403; 4. Compare password against `password_hash` using `bcrypt.compare()`; 5. On match: generate signed JWT containing `userId`, `username`, `role`, `jamiat_id`, `jamaat_id`, `executive_level`; token valid for 24 hours; 6. On failure: return HTTP 401 without revealing whether username or password was incorrect
Output	HTTP 200: `{ token, user: { id, username, role, full_name, permissions[] } }`; HTTP 401 on failure
Rate Limiting	Maximum 5 failed attempts per IP per 15 minutes via `express-rate-limit`
Database	SELECT from `users` WHERE username = ? OR email = ?

SRS-AUTH-002: JWT Verification Middleware

ATTRIBUTE	DETAIL
Priority	Critical
Trigger	Every request to a protected API endpoint
Processing	1. Extract Bearer token from `Authorization` header; 2. Verify JWT signature using `JWT_SECRET`; 3. Check token expiry; 4. Re-fetch user from DB to validate `is_active` status; 5. Attach decoded user object to `req.user`; 6. Reject with HTTP 401 if any check fails
Output	Request proceeds with `req.user` populated; HTTP 401 if unauthorized

SRS-AUTH-003: RBAC Permission Middleware

ATTRIBUTE	DETAIL
Priority	Critical
Trigger	After JWT verification, on every protected endpoint that specifies a required permission
Processing	1. Resolve user's effective permissions from all assigned roles in `user_roles` junction table; 2. Check if required `resource:action` string exists in the resolved permission set; 3. Super Admin bypasses all permission checks; 4. Reject with HTTP 403 if permission not present
Permission Format	`resource:action` — e.g., `cases:read`, `cases:create`, `counseling_forms:update`, `reports:export`
Output	Request proceeds if permitted; HTTP 403 Forbidden if not

SRS-AUTH-004: Jamiat/Jamaat Scope Filter

ATTRIBUTE	DETAIL
Priority	Critical
Trigger	Any list/search query for cases, applicants, or users
Processing	1. Inspect `req.user.role` and `req.user.jamiat_id` ; 2. If role is Super Admin or Admin: no WHERE clause injected; 3. If role is DCM or Welfare Reviewer: inject `WHERE cases.jamiat_id = req.user.jamiat_id` ; 4. If role is Counselor: inject `WHERE cases.jamaat_id = req.user.jamaat_id` ; 5. Scope constraint applied at SQL query level — not just frontend filtering
Output	Result set filtered to authorized organizational scope only

SRS-AUTH-005: Profile Retrieval and Permission Refresh

ATTRIBUTE	DETAIL
Priority	High
Endpoint	`GET /api/auth/profile`
Processing	1. Validate JWT token; 2. Fetch full user profile from DB including all assigned roles and per-role Jamiat/Jamaat assignments; 3. Resolve and return complete permission tree to React frontend for UI rendering decisions
Output	Full user profile JSON with `assigned_roles[]` and flattened `permissions[]`

SRS-AUTH-006: Logout

ATTRIBUTE	DETAIL
Priority	High
Processing	JWT logout is client-side token discard (stateless architecture). Server provides `POST /api/auth/logout` endpoint for audit logging purposes. No server-side session state is maintained.

3.2 User Management

3.2.1 Description

The user management module allows administrators to create, view, update, deactivate, and export user accounts across all 8 roles. It supports individual and bulk operations including Excel import with ITS API enrichment.

3.2.2 Functional Requirements

SRS-USER-001: Create User

ATTRIBUTE	DETAIL
Priority	Critical
Access	Admin, Super Admin
Input	`full_name` (required), `username` (required, unique), `email` (required, unique), `role` (required), `password` (optional — defaults to `TempPassword123!`), `phone` , `its_number` , `photo` (base64), `jamiat_id` , `jamaat_id` , `executive_level` , `role_jamiat_jamaat[]`
Processing	1. Validate all required fields; 2. Enforce username and email uniqueness across `users` table; 3. Hash password using `bcryptjs` (10 rounds); 4. INSERT into `users` ; 5. If `role_jamiat_jamaat[]` provided, INSERT per-role scope into `user_roles` ; 6. Log creation in audit trail
Output	HTTP 201: `{ message, userId }` ; HTTP 409 on duplicate username/email
Database	INSERT `users` , INSERT `user_roles` (if multi-role)

SRS-USER-002: Update User

ATTRIBUTE	DETAIL
Priority	High
Access	Admin, Super Admin
Input	`user_id` , any updatable field; `password` change is optional (re-hashed if provided)
Processing	1. Validate fields; 2. Enforce uniqueness on username/email excluding current user; 3. UPDATE `users` ; 4. Replace `user_roles` entries if roles/scope provided; 5. Log update in audit trail
Output	HTTP 200: `{ message: "User updated successfully" }`

SRS-USER-003: Activate / Deactivate User

ATTRIBUTE	DETAIL
Priority	High
Access	Admin, Super Admin
Processing	PATCH `users` SET `is_active` = 0 or 1; deactivated users rejected at JWT verification middleware mid-session; account data preserved for audit trail

SRS-USER-004: List and Filter Users

ATTRIBUTE	DETAIL
Priority	High
Access	Admin, Super Admin
Input	Query params: `page` , `limit` (10/20/50/100/500), `role` , `is_active` , `search` (name/email/username), `jamiat_id`
Processing	SELECT from `users` JOIN `user_roles` with pagination and filter WHERE clauses; return total count and paginated data
Output	`{ total, page, limit, data: [ { id, username, role, assigned_roles[], is_active } ] }`

SRS-USER-005: Bulk Import via Excel

ATTRIBUTE	DETAIL
Priority	Medium
Access	Admin, Super Admin
Input	Excel file (.xlsx) matching downloadable template from `GET /api/users/export/template`
Processing	1. Parse file using ExcelJS; 2. Validate each row; 3. Check username/email uniqueness; 4. INSERT valid rows with hashed passwords; 5. Optionally call ITS API for users with ITS numbers to enrich profiles; 6. Return row-level results: inserted/updated/skipped/errors
Output	Import summary with success count and per-row error details

SRS-USER-006: Export Users to Excel

ATTRIBUTE	DETAIL
Priority	Medium
Access	Admin, Super Admin
Processing	Generate Excel file via ExcelJS with all user records matching applied filters; stream as download with Content-Disposition header
Output	Downloaded `users_export_{date}.xlsx`

3.3 Case Identification and Eligibility Screening

3.3.1 Description

The case identification module is the intake gateway for all welfare applicants. It integrates with the external ITS API for automated demographic prefill, enforces duplicate detection, captures income assessment fields, and governs the transition to formal case creation upon an eligible decision.

3.3.2 Functional Requirements

SRS-ID-001: Create Case Identification

ATTRIBUTE	DETAIL
Priority	Critical
Access	DCM, Admin, Super Admin
Input	`its_number` (8-digit, required), `eligible_in` (case_type_id, required), `total_family_members` (0–25), `earning_family_members` (0–20), `individual_income` (0–9,999,999), `family_income` (0–9,999,999), `remarks` (optional), Jamiat/Jamaat assignment
Processing	1. Validate ITS number format (8 numeric digits); 2. Query `applicants` table — if found, link; else trigger ITS API lookup and create applicant record; 3. Check for active duplicate case on same ITS number (block if active case exists in non-closed/non-rejected state with HTTP 409); 4. INSERT into `case_identifications` with status = 'pending'; 5. Log income baseline
Output	HTTP 201: `{ id, its_number, status: "pending", applicant: {...} }`
Database	INSERT `case_identifications` ; INSERT `applicants` if new

SRS-ID-002: ITS Number Lookup (Auto-Fetch)

ATTRIBUTE	DETAIL
Priority	Critical
Endpoint	`GET /api/applicants/lookup/:itsNumber`
Processing	1. Validate 8-digit numeric format; 2. Query local `applicants` table first (cache-first strategy); 3. If not found locally, proxy request to external ITS API with masked credentials in Node server; 4. Parse response capturing: `full_name` , `age` , `gender` , `phone` , `email` , `photo` (base64), `address` , `jamiat` , `jamaat` ; 5. Cache result in `applicants` table; 6. Return formatted payload to frontend for auto-population
Error Handling	ITS API timeout (5 seconds): return graceful empty response; network error: log and return HTTP 503 with user-friendly message
Output	HTTP 200: populated applicant object; HTTP 404: applicant not found

SRS-ID-003: Review — Eligible or Ineligible

ATTRIBUTE	DETAIL
Priority	Critical
Access	Permission: `case_identification:approve`
Input	`identification_id` , `decision` ('eligible' or 'ineligible'), `review_remarks` (required on ineligible)
Processing	Eligible path: 1. UPDATE `case_identifications` SET status = 'eligible'; 2. Auto-generate case number (e.g., BS-0001, sequential per case type); 3. INSERT into `cases` table with status = 'Draft', linked to applicant; 4. Auto-create `counseling_forms` record linked to new case; 5. Send notification. Ineligible path: 1. UPDATE status = 'ineligible'; 2. Store review_remarks, reviewed_by, reviewed_at; 3. Send notification
Output	HTTP 200: `{ message, caseId, case_number }` (eligible); HTTP 200: `{ message }` (ineligible)

SRS-ID-004: Edit Assessment Fields with Income Log

ATTRIBUTE	DETAIL
Priority	High
Access	Permission: `case_identification:edit`
Input	`individual_income` , `family_income` , `eligible_in` , `total_family_members` , `earning_family_members` , `remarks`
Processing	1. UPDATE `case_identifications` ; 2. For each income field change: INSERT into `case_identification_income_logs` (identification_id, field_name, old_value, new_value, changed_by, changed_at); 3. Full audit trail maintained for all income modifications
Output	HTTP 200: `{ message: "Case identification updated" }`

SRS-ID-005: Revert Ineligible to Pending

ATTRIBUTE	DETAIL
Priority	Medium
Access	Permission: `case_identification:edit`
Processing	1. Validate current status is 'ineligible'; 2. UPDATE status = 'pending'; 3. Clear reviewed_by, reviewed_at, review_remarks; 4. Log revert action in audit trail

---

3.4 Case Management and Lifecycle Tracking

3.4.1 Description

The core case management module handles the full lifecycle of each welfare case from creation through closure. It manages assignments, workflow stage transitions, comments, SLA tracking, and maintains a complete status history audit trail.

3.4.2 Functional Requirements

SRS-CASE-001: Create Case

ATTRIBUTE	DETAIL
Priority	Critical
Access	Admin, DCM (auto-triggered from eligible case identification)
Input	`applicant_id` (or inline `applicant_data`), `case_type_id` , optional: `assigned_dcm_id` , `assigned_counselor_id` , `jamiat_id` , `jamaat_id`
Processing	1. Generate case number (sequential per case type, zero-padded — e.g., BS-0001); 2. INSERT into `cases` with status = first workflow stage for case type; 3. Auto-create linked `counseling_forms` record; 4. Record stage entry timestamp in `cases.current_stage_entered_at` ; 5. Notify assigned counselor
Output	HTTP 201: `{ message, caseId, case_number }`
Database	INSERT `cases` , INSERT `counseling_forms` , INSERT `case_status_history`

SRS-CASE-002: Assign Case

ATTRIBUTE	DETAIL
Priority	Critical
Access	Admin, DCM
Input	`case_id` , `counselor_id` (filtered to Counselor role within Jamaat scope)
Processing	1. Validate counselor has Counselor role in matching Jamaat; 2. UPDATE `cases.assigned_counselor_id` ; 3. Create in-app notification for assigned counselor; 4. Log assignment action in `workflow_logs`
Output	HTTP 200: `{ message: "Case assigned successfully" }`

SRS-CASE-003: Case Comments

ATTRIBUTE	DETAIL
Priority	High
Access	All roles with `cases:read` permission
Input	`case_id`, `comment_text` (required), `comment_type` (general / rejection / rework)
Processing	INSERT into `case_comments` (case_id, user_id, comment_text, comment_type, created_at)
Output	Comment thread displayed chronologically on case detail view

SRS-CASE-004: SLA Tracking

ATTRIBUTE	DETAIL
Priority	High
Trigger	Case stage entry; background cron job (configurable interval)
Processing	1. On stage entry: record `cases.current_stage_entered_at = NOW()`; 2. Cron evaluates all active cases: compute elapsed hours = (NOW() - current_stage_entered_at); 3. Compare against `workflow_stages.sla_hours`; 4. UPDATE `cases.sla_status`: 'on_time' (< 80% of SLA), 'at_risk' (80–99%), 'breached' (≥ 100%); 5. Trigger notification to assigned counselor on 'at_risk'; 6. Trigger escalation notification on 'breached'
Visual Indicator	SLA status badge color rendered using `workflow_stages.hex_color`

SRS-CASE-005: Case Closure

ATTRIBUTE	DETAIL
Priority	High
Access	Admin, DCM
Input	`case_id`, `closure_reason` (required), `closure_type` (completed / withdrawn / ineligible)
Processing	1. Validate case is not already closed; 2. UPDATE `cases` SET status = 'Closed', `closed_at`, `closed_by`, `closure_reason`; 3. INSERT final entry into `case_status_history`; 4. Notify assigned counselor

SRS-CASE-006: Case Search and Filtering

ATTRIBUTE	DETAIL
Priority	High
Access	All authenticated roles (scoped by Jamiat/Jamaat)
Input	Query params: `page`, `limit`, `status`, `case_type_id`, `jamiat_id`, `jamaat_id`, `counselor_id`, `search` (case number / applicant name), `sla_status`, `date_from`, `date_to`
Processing	SELECT from `cases` JOIN `applicants` JOIN `workflow_stages` with scope injection and filter WHERE clauses; paginated response
Output	Paginated list with total count and case summary objects

---

3.5 Counseling Forms — Digital Assessment (8 Sections)

3.5.1 Description

The counseling form module is an 8-section digital wizard that replaces paper-based counseling documentation. Each section is saved independently, allowing partial saves without losing progress. The form is completed by the assigned Counselor and validated for completeness before the case can advance beyond the Counseling workflow stage.

3.5.2 Functional Requirements

SRS-FRM-001: Section 1 — Personal Details

ATTRIBUTE	DETAIL
Priority	Critical
Access	Counselor, DCM, Admin
Processing	1. Auto-populated from ITS API data at case creation; 2. Editable fields: name, address, phone, occupation, education_level; 3. PATCH `/api/counseling-forms/:id/section/personal` ; 4. UPSERT into `personal_details` ; 5. Update `counseling_forms.personal_details_id`

SRS-FRM-002: Section 2 — Family Details

ATTRIBUTE	DETAIL
Priority	Critical
Processing	1. Dynamic table for family members (1-to-many); each member: name, relation (FK to `relations`), age, occupation (FK to `occupations`), income, education (FK to `education_levels`); 2. Aggregate calculated: `total_monthly_income` , `total_liabilities` ; 3. PATCH `/api/counseling-forms/:id/section/family` ; 4. UPSERT `family_details` header; 5. DELETE + INSERT `family_members` for current case

SRS-FRM-003: Section 3 — Assessment

ATTRIBUTE	DETAIL
Priority	Critical
Processing	1. Captures background narrative, education history, health conditions, social circumstances; 2. Multi-select `assessment_products_services[]` for proposed business or service; 3. PATCH `/api/counseling-forms/:id/section/assessment` ; 4. UPSERT `assessment` table; 5. Handle array fields via JSON serialization

SRS-FRM-004: Section 4 — Financial Assistance

ATTRIBUTE	DETAIL
Priority	Critical
Processing	1. Captures: `self_funding` , `rahen_available` (collateral), `requested_enayat` , `requested_qardan` , `recommended_enayat` , `recommended_qardan` ; 2. `qh_fields` stored as JSON for flexible repayment schedule schemas; 3. Validates: total assistance ≥ self_funding; 4. PATCH `/api/counseling-forms/:id/section/financial` ; 5. UPSERT `financial_assistance`

SRS-FRM-005: Section 5 — Economic Growth Projections

ATTRIBUTE	DETAIL
Priority	High
Processing	1. Multi-year financial projection table: last_year + years 1–5 columns for: revenue_sales, total_expenses, profit, cash_in_hand, stock_equipment; 2. Frontend auto-calculates `profit_yearN = revenue_yearN - expenses_yearN` ; 3. PATCH `/api/counseling-forms/:id/section/economic-growth` ; 4. UPSERT `economic_growth` with all year columns as DECIMAL(15,2)

SRS-FRM-006: Section 6 — Declaration and Digital Signatures

ATTRIBUTE	DETAIL
Priority	Critical
Processing	1. Captures declaration text acknowledgment checkbox; 2. Canvas-drawn digital signatures for: applicant, counselor, TR committee; 3. Signatures stored as base64 data URI strings in `declaration` table columns: `applicant_signature` , `counselor_signature` , `committee_signature` ; 4. PATCH `/api/counseling-forms/:id/section/declaration`

SRS-FRM-007: Section 7 — Attachments

ATTRIBUTE	DETAIL
Priority	High
Processing	1. Multi-file upload per attachment category (ID documents, income proof, photos); 2. Files uploaded via `POST /api/attachments` with `case_id` and `section` = 'counseling'; 3. Records stored in `case_attachments` ; 4. Downloadable from `GET /api/attachments/:id/download`

SRS-FRM-008: Section 8 — Manzoori (Final Approval Notes)

ATTRIBUTE	DETAIL
Priority	High
Processing	1. Captures final counselor approval notes and recommendations; 2. DCM review comments; 3. Proposed assistance summary; 4. PATCH `/api/counseling-forms/:id/section/manzoori` ; 5. Setting `counseling_forms.is_complete = 1` requires all 8 section FK pointers to be non-null; 6. `is_complete` is a prerequisite for workflow advancement from Counseling stage

---

3.6 Multi-Level Workflow and Approvals

3.6.1 Description

The workflow engine manages case progression through configurable stages. Each case type has its own ordered stage sequence. Role-gated transitions enforce that only authorized users at the correct stage can advance, reject, or send back cases. All transitions are logged in the status history for full auditability.

3.6.2 Functional Requirements

SRS-WF-001: Workflow Stage Progression

ATTRIBUTE	DETAIL
Priority	Critical
Endpoint	`PUT /api/cases/:id/workflow-action`
Input	`action` (forward / reject / rework / resubmit), `comments` (required for reject/rework), `target_stage_id` (optional for rework to specific stage)
Processing	1. Load current case and `workflow_stages` sequence for case type; 2. Validate requesting user's role against `workflow_stages.role_name` for current stage; 3. Validate prerequisites: for Counseling→Welfare Review: `counseling_forms.is_complete = 1` ; for Welfare Review→ZI: welfare checklist complete; 4. Forward: UPDATE `cases.status_id` to next stage; record `current_stage_entered_at = NOW()` ; 5. Reject: UPDATE status to 'Rejected'; 6. Rework: UPDATE status to previous stage or target stage; require rework comment; 7. INSERT into `case_status_history` and `workflow_logs` ; 8. Trigger notification to next stage's responsible user
Output	HTTP 200: `{ message, new_stage }`

SRS-WF-002: Workflow Stage Definitions (Baaseteen Model)

STAGE	REQUIRED ROLE	PREREQUISITES	ACTIONS AVAILABLE
Draft	DCM, Admin	None	Forward to Counseling; Reject; Close
Counseling	Counselor	Counselor assigned	Forward (requires is_complete=1); Rework
Welfare Review	Welfare Reviewer	Counseling form complete; welfare checklist complete	Approve; Reject; Rework to Counseling
ZI Approval	ZI / ZI Core	Welfare Review approved	Approve; Rework; Reject
KG Review	KG Approver	ZI Approval	Approve; Rework; Reject

STAGE	REQUIRED ROLE	PREREQUISITES	ACTIONS AVAILABLE
Operations Lead	Operations Approver	KG Review	Approve; Rework; Reject
Cover Letter Edit	Admin / DCM	Operations Lead approved	Generate Cover Letter; Forward
Executive Approval (1–5)	Executive (by level)	Sequential from Cover Letter	Approve; Reject; Forward to next executive level
Manzoori (Sanction)	Admin	All executive tiers approved	Sanction; Reject
Financial Disbursement	Finance	Manzoori sanctioned	Confirm installments; Close case

SRS-WF-003: Status History Audit Trail

ATTRIBUTE	DETAIL
Priority	Critical
Processing	Every workflow action INSERTs into `case_status_history` : (case_id, from_stage_id, to_stage_id, action, performed_by, comments, timestamp). Records are immutable and cannot be deleted or modified.

3.7 Cover Letter Generation

3.7.1 Description

The cover letter module consolidates all case data into a standardized cover letter document and generates a downloadable PDF using Puppeteer. It captures welfare department digital signatures and supports iterative edits before final PDF generation.

3.7.2 Functional Requirements

SRS-COV-001: Cover Letter Form Completion

ATTRIBUTE	DETAIL
Priority	High
Access	Admin, DCM (at Cover Letter Edit workflow stage)
Processing	1. Pre-populate cover letter form by aggregating: applicant demographics, counselor details, financial assistance figures (recommended and approved amounts), economic growth summary, and workflow history; 2. Allow editing of: `welfare_department_signature_drawing_data` (canvas-drawn base64 signature), `approved_enayat` , `approved_qardan` (executive override values), narrative remarks; 3. UPSERT into `cover_letter_forms` table; 4. Mark `is_complete = 1` when all required fields are present

SRS-COV-002: PDF Generation

ATTRIBUTE	DETAIL
Priority	High
Endpoint	`POST /api/cover-letters/generate/:caseId`
Processing	1. Fetch complete `cover_letter_forms` data joined with all related entities; 2. Render into standardized HTML template with inline CSS; 3. Launch Puppeteer headless Chrome instance; 4. Generate PDF (A4, margins: 15mm); 5. Save to `UPLOAD_PATH/cover_letters/{caseId}_{timestamp}.pdf` ; 6. Store file path in `cover_letter_forms.pdf_path` ; 7. Return download URL
Output	HTTP 200: `{ pdfUrl, message }` ; PDF streamed to browser on download request

---

3.8 Payment and Financial Management

3.8.1 Description

After Manzoori sanction, approved financial assistance amounts are converted into structured disbursement schedules. The Finance team confirms actual disbursements on individual installments, maintaining a real-time financial ledger per case.

3.8.2 Functional Requirements

SRS-PAY-001: Generate Payment Schedule

ATTRIBUTE	DETAIL
Priority	Critical
Trigger	Manzoori sanction confirmed; `approved_enayat` and `approved_qardan` finalized in `cover_letter_forms`
Processing	1. Load `financial_assistance.qh_fields` (JSON repayment configuration) and approved amounts; 2. Parse disbursement schedule: Enayat as lump-sum or phased grants; Qardan as installment repayments over 1–5 years; 3. INSERT installment rows into `repayment_schedules` (case_id, installment_number, amount_due, due_date, type [enayat/qardan], is_paid = 0); 4. Calculate total commitment
Output	Structured installment rows visible to Finance team on case detail

SRS-PAY-002: Confirm Disbursement

ATTRIBUTE	DETAIL
Priority	High
Access	Finance role only
Endpoint	`POST /api/cases/:caseId/payment-schedule/:id/confirm-disbursement`
Processing	1. Validate requesting user has Finance role; 2. Validate installment belongs to specified case; 3. UPDATE `repayment_schedules` SET `is_paid = 1`, `paid_at = NOW()`, `confirmed_by = req.user.id`; 4. Recalculate `cases.total_disbursed`
Output	HTTP 200: `{ message: "Disbursement confirmed" }`

SRS-PAY-003: View Payment Summary

ATTRIBUTE	DETAIL
Priority	High
Access	Finance, Admin, DCM
Processing	Aggregate all installments for a case: total approved, total disbursed, total pending, overdue installments; display per-installment timeline

---

3.9 Fund Utilization and Quarterly Reporting

3.9.1 Description

Fund utilization reports are quarterly reassessments verifying that disbursed funds are being used as intended. The reporting form is dynamically generated from admin-configured question schemas, allowing new question types and sections to be added without code changes.

3.9.2 Functional Requirements

SRS-FUN-001: Dynamic Form Schema

ATTRIBUTE	DETAIL
Priority	High
Access	Admin (schema configuration); Counselor (form completion)

ATTRIBUTE	DETAIL
Processing	1. Admin defines sections in `fund_utilization_sections` (title, sort_order); 2. Admin defines questions per section in `fund_utilization_questions` : `field_type` (text/number/select/file_upload/table), `label` , `is_required` , `options_json` (for selects), `auto_fill_key` (for pre-population); 3. Frontend `GET /api/fund-utilization-master/sections` renders React components dynamically per field_type; 4. Validation rules (`min_length` , `max_length` , `is_required`) enforced at submission

SRS-FUN-002: Submit Quarterly Report

ATTRIBUTE	DETAIL
Priority	Critical
Input	`case_id` , `quarter` (q1/q2/q3/q4), `year` , `responses_json` (key-value map of question_id → answer), photo uploads
Processing	1. Validate required questions answered; 2. Process file uploads via Multer — only JPEG/PNG allowed; 3. INSERT/UPDATE `fund_utilization_reports` (case_id, quarter, year, responses_json, submitted_by, submitted_at); 4. INSERT photo records into `fund_utilization_photos` (report_id, photo_type, file_path)
Output	HTTP 201: `{ message, reportId }`

SRS-FUN-003: Generate Fund Utilization PDF

ATTRIBUTE	DETAIL
Priority	High
Processing	1. Fetch report + question schema + applicant data; 2. Render HTML template with question/answer pairs and embedded photos; 3. Generate PDF via Puppeteer; 4. Store and return download URL

3.10 Welfare Checklist

3.10.1 Description

The welfare checklist enforces a structured review gate before cases can advance past the Welfare Review stage. Admins configure checklist categories and items; Welfare Reviewers mark items complete per case.

3.10.2 Functional Requirements

SRS-CHK-001: Manage Checklist Configuration

ATTRIBUTE	DETAIL
Priority	High
Access	Admin, Super Admin
Processing	CRUD on `welfare_checklist_categories` (title, sort_order) and `welfare_checklist_items` (category_id, item_text, is_required, sort_order); drag-and-drop sort_order update via PATCH endpoint

SRS-CHK-002: Complete Checklist Per Case

ATTRIBUTE	DETAIL
Priority	Critical
Access	Welfare Reviewer
Processing	1. Load all checklist items for case type; 2. Welfare Reviewer marks each item as complete/incomplete with optional notes; 3. UPSERT into `welfare_checklist_responses` (case_id, item_id, status, notes, reviewed_by, reviewed_at); 4. Checklist gate: workflow action to advance from Welfare Review blocked until all `is_required = 1` items are marked complete

---

3.11 Document Management and Attachments

3.11.1 Description

Files are attached to cases and individual counseling sections throughout the case lifecycle. All uploads are validated for type and size, stored on the local filesystem, and tracked in the database for access and download.

3.11.2 Functional Requirements

SRS-DOC-001: Upload Attachment

ATTRIBUTE	DETAIL
Priority	High
Access	All roles with `cases:update` permission
Input	`case_id` (required), `section` (optional: counseling / payment / fund_utilization), `document_type`, file (multipart/form-data)
Validation	Allowed types: PDF, DOC, DOCX, JPG, JPEG, PNG; Max size: 5MB per file; Max 10 files per upload batch
Processing	1. Multer receives file stream; 2. Validate MIME type and size; 3. Save to `UPLOAD_PATH/cases/{caseId}/{uuid}_{original_name}` ; 4. INSERT into `case_attachments` (case_id, section, document_type, file_name, file_path, file_size, uploaded_by, uploaded_at)
Output	HTTP 201: `{ message, attachmentId, downloadUrl }`

SRS-DOC-002: Download Attachment

ATTRIBUTE	DETAIL
Priority	High
Processing	1. Validate user has access to the case the attachment belongs to (scope check); 2. Stream file from filesystem with `Content-Disposition: attachment` header; 3. Log download in audit trail

SRS-DOC-003: Delete Attachment

ATTRIBUTE	DETAIL
Priority	Medium
Access	Admin, DCM, or uploader (own files only)
Processing	1. Validate ownership or admin role; 2. DELETE file from filesystem; 3. DELETE record from `case_attachments` ; 4. Log deletion in audit trail

---

3.12 Notification System

3.12.1 Description

The notification system delivers in-app alerts for key case events and SLA breaches. Notifications are stored per user and displayed with unread count badges in the navigation header. Optional SMTP email delivery is configurable.

3.12.2 Functional Requirements

SRS-NOTIF-001: Auto-Create Notifications

ATTRIBUTE	DETAIL
Priority	High
Trigger Events	

EVENT	RECIPIENTS
Case identification eligible decision	Assigned DCM

EVENT	RECIPIENTS
Case assigned to counselor	Assigned Counselor
Case advanced to next workflow stage	Responsible user for next stage
Case rejected / rework requested	Case's assigned counselor and DCM
SLA at_risk threshold reached	Assigned counselor, DCM
SLA breached	Assigned counselor, DCM, Admin
Payment disbursement confirmed	Assigned DCM
Fund utilization report submitted	DCM, Admin

Storage: INSERT INTO `notifications` (user_id, case_id, title, message, type, is_read = 0, created_at)

SRS-NOTIF-002: View Notifications

ATTRIBUTE	DETAIL
Priority	High
Access	All authenticated users
Output	Paginated notification list with: title, message, type badge, relative timestamp, read/unread styling; link to related case

SRS-NOTIF-003: Unread Badge

ATTRIBUTE	DETAIL
Priority	Medium
Implementation	Header navigation badge shows COUNT of `is_read = 0` notifications for current user; React Query polling (30-second interval) for real-time updates

SRS-NOTIF-004: Mark as Read

ATTRIBUTE	DETAIL
Priority	Medium
Processing	UPDATE `notifications` SET is_read = 1, read_at = NOW() WHERE id = ? (individual) or WHERE user_id = ? (mark all read)

3.13 Dashboard Analytics and Reporting

3.13.1 Description

Role-specific dashboards present real-time pipeline statistics filtered by the authenticated user's organizational scope. Reporting includes Excel export for bulk data access.

3.13.2 Functional Requirements

SRS-DASH-001: Dashboard Statistics

ATTRIBUTE	DETAIL
Priority	High
Endpoint	`GET /api/dashboard/stats`
Processing	1. Inject organizational scope filters per requesting user's role; 2. Execute aggregate SQL queries counting cases per status; 3. Count: total cases, active cases, pending review, closed this month, SLA at_risk/breached; 4. Return recent activity feed (last 10 case actions by user's scope)

ROLE	DASHBOARD FOCUS
Super Admin / Admin	System-wide totals, all pipeline stages, user activity summary
DCM	Jamiat-scoped case pipeline, counselor workload, SLA alerts
Counselor	Assigned cases, form completion status, pending actions
Finance	Pending disbursements, overdue installments, payment totals
Executive	Pending approval queue at their executive level, recent approvals

SRS-DASH-002: Pipeline Analytics

ATTRIBUTE	DETAIL
Priority	Medium
Processing	Bar/pie chart data for cases by workflow stage; trend data for cases created vs. closed per month; consumed by Recharts components on the React dashboard

SRS-RPT-001: Excel Export

ATTRIBUTE	DETAIL
Priority	High
Access	Admin, Super Admin, Report Viewer
Supported Exports	Cases list with all fields; Applicants list; Users list; Payment schedules; Fund utilization summaries
Processing	1. Apply same scope filters as list endpoints; 2. Generate XLSX using ExcelJS with header row and formatted data; 3. Stream with Content-Disposition: attachment

---

3.14 Master Data Management

3.14.1 Description

Master data defines the structural foundation of the system — organizational hierarchy, case types, workflow stages, and dropdown dictionaries. Changes to master data affect all users and must enforce dependency protection.

3.14.2 Functional Requirements

SRS-MST-001: Manage Jamiat and Jamaat

ATTRIBUTE	DETAIL
Priority	Critical
Access	Super Admin, Admin
Processing	CRUD on `jamiat` (name, jamiat_id code, is_active) and `jamaat` (name, jamaat_id code, jamiat_id FK, is_active); deletion blocked via HTTP 409 if active cases or users are scoped to the target record

SRS-MST-002: Manage Workflow Stages

ATTRIBUTE	DETAIL
Priority	Critical
Access	Super Admin
Processing	CRUD on `workflow_stages`: name, description, case_type_id (FK), sort_order (controls sequence), role_name (which role can act), sla_hours (default 48), hex_color (UI badge color); sort_order must be unique per case_type_id

SRS-MST-003: Manage Executive Levels

ATTRIBUTE	DETAIL
Priority	High
Processing	CRUD on `executive_levels` : level_number (1–5, unique), level_name, fund_threshold (max amount this level can approve); affects which Executive users appear in approval queues for cases at each executive workflow stage

SRS-MST-004: Manage Dropdown Dictionaries

ATTRIBUTE	DETAIL
Priority	High
Tables	`relations` (family kinship: Father, Mother, Son, etc.), `education_levels` (Illiterate, Primary, Secondary, etc.), `occupations` (Business, Employee, Unemployed, etc.), `case_types` (Baaseteen, SHND, etc.)
Processing	CRUD with is_active flag; deletion blocked if records are referenced in counseling form data

4. External Interface Requirements

4.1 User Interfaces

4.1.1 Global Layout

```
┌──────────────────────────────────────────────────────────────────────┐
│  TOP HEADER                                                          │
│  ┌────────┐  ┌──────────────────────────────────┐  ┌─────────┐       │
│  │  Logo  │  │  Organization Title              │  │ 🔔  N   │ │
│  └────────┘  └──────────────────────────────────┘  │ Profile│ │
│                                                      └─────────┘ │
├──────────────┼───────────────────────────────────────────────────────┤
│  LEFT        │                                                       │
│  SIDEBAR     │  CONTENT AREA                                         │
│              │                                                       │
│  Role-based  │  ┌──────────────────────────────────────────────┐    │
│  navigation  │  │  Breadcrumb / Page Title                     │    │
│  with icons  │  ├──────────────────────────────────────────────┤    │
│              │  │                                              │    │
│  Dashboard   │  │  Page Content: Tables, Forms, Cards,         │    │
│  Cases       │  │  Modals, Charts, Workflow Progress           │    │
│  Applicants  │  │                                              │    │
│  Reports     │  └──────────────────────────────────────────────┘    │
│  Master Data │                                                       │
│  ...         │  ┌──────────────────────────────────────────────┐    │
│              │  │  Toast Notifications (bottom-right)          │    │
│              │  └──────────────────────────────────────────────┘    │
└──────────────┴───────────────────────────────────────────────────────┘
```

4.1.2 Login Page (`/login`)

ELEMENT	SPECIFICATION
Layout	Centered card on full-screen background; responsive for mobile and desktop
Fields	Username and Password inputs with show/hide password toggle
Button	"Login" with loading spinner; submits JWT credential request
Error Display	Alert banner above the form for authentication failures
Token Storage	JWT stored in AuthContext (in-memory); user redirected to Dashboard on success

4.1.3 Dashboard (`/dashboard`)

ROLE	LAYOUT
Admin / Super Admin	Total cases, pending, active, closed stat cards; organization-wide pipeline chart; recent activity feed
DCM	Jamiat-scoped case counts; cases by stage bar chart; workload distribution across counselors
Counselor	Assigned cases with form completion progress indicators; SLA status badges
Executive / ZI	Cases awaiting their specific approval; priority queue sorted by SLA status
Finance	Pending disbursements; payment schedule calendar view; confirmed vs pending counts

4.1.4 Case Details Tabbed Layout (`/cases/:id`)

TAB	CONTENT
Overview	Applicant info, case type, current stage, SLA status badge, workflow history timeline, action buttons
Counseling Form	8-section accordion/tab form with section-level save and completion checkmarks
Cover Letter	Cover letter form with pre-populated data; Generate PDF button; previously generated PDFs listed

TAB	CONTENT
Payments	Qardan Hasana and Enayat payment schedule; disbursement confirmation controls for Finance role
Fund Utilization	Per-quarter dynamic form based on admin-configured schema; photo upload; quarterly PDF export
Welfare Checklist	Checklist items grouped by category; Y/N/NA radio buttons with optional comment per item
Attachments	File upload area with drag-and-drop; stage-tagged attachment list with download/delete
Comments	General comments thread; workflow comments per stage transition visible chronologically

4.1.5 Workflow Progress Indicator

ELEMENT	SPECIFICATION
Stage Steps	Horizontal step bar showing all workflow stages; current stage highlighted; completed stages marked
SLA Badges	Color-coded pills per case: green (on_time), amber (at_risk), red (breached)
Action Buttons	Role-conditional forward/reject/rework buttons visible only when user has permission for current stage
Comments Required	Rework and reject actions require a mandatory comment before submission

4.2 Hardware Interfaces

The system has no direct hardware interfaces. All interaction is through standard web browsers on devices with screen and internet connectivity. The signature pad component supports both mouse/stylus drawing and file upload modes for capturing digital signatures.

4.3 Software Interfaces

INTERFACE	PROTOCOL	PURPOSE
MySQL 8.0	TCP port 3306 (mysql2)	Primary data storage; connection pool of 10 persistent connections
Node.js v16+ / Express	HTTP/HTTPS	RESTful API server; 150+ endpoints across 25 route modules
React 18.2 (Vite)	Browser / CDN	SPA frontend; Axios HTTP client with JWT interceptors
ITS External API	HTTPS REST	Applicant demographic data fetch; 5-second timeout with graceful fallback
PDFKit	Node.js library	Programmatic PDF generation for cover letters
Puppeteer	Headless Chromium	HTML → PDF rendering for quarterly fund utilization reports
Multer	Node.js middleware	Multipart/form-data file upload handling; MIME validation; 10 MB size limit
Nodemailer	SMTP (TLS port 587)	Optional email notification dispatch; configurable via .env
Helmet.js	Express middleware	HTTP security headers (CSP, X-Frame-Options, HSTS)
Tailwind CSS v3.4	CSS framework	Utility-based responsive styling; custom theme configuration

4.4 Communication Interfaces

INTERFACE	PROTOCOL	DESCRIPTION
HTTP/HTTPS	TCP 80/443	All client–server communication; Nginx handles SSL termination in production
JSON REST API	Axios (XMLHttpRequest)	All data exchange between React SPA and Express backend; standard HTTP methods and status codes
JWT Bearer Token	Authorization Header	Stateless authentication credential injected on every API request by Axios interceptors
Multipart Form Data	HTTP POST	File uploads (attachments, signatures, fund utilization photos) via Multer
SMTP TLS	TCP 587	Email notifications via Nodemailer when email is configured in .env

INTERFACE	PROTOCOL	DESCRIPTION
ITS API	HTTPS GET	External ITS service call for applicant demographic pre-fill; proxied through backend to mask credentials

5. Non-Functional Requirements

5.1 Performance Requirements

ID	REQUIREMENT	TARGET	MEASUREMENT
PR-001	Standard API response (list endpoints)	< 500 ms at p95	Server processing time (paginated, indexed)
PR-002	Case detail fetch (full — 10+ table JOINs)	< 800 ms	Server processing time
PR-003	Dashboard overview (role-filtered aggregates)	< 1 second	Time to first meaningful render
PR-004	ITS external API lookup	< 3 seconds	Network-dependent; graceful timeout enforced
PR-005	File upload processing (10 MB)	< 10 seconds	Multer streaming to filesystem
PR-006	Cover letter PDF generation (PDFKit)	< 5 seconds	PDF generation + stream to client
PR-007	Quarterly report PDF (Puppeteer)	< 8 seconds	Headless browser render + save
PR-008	Excel bulk import (100 applicant rows)	< 5 seconds	Validation + DB insert time
PR-009	SLA check across all active cases	< 3 seconds	Batch calculation on active case set
PR-010	Concurrent user sessions supported	50+ simultaneous	Node.js async I/O; connection pool

5.2 Safety Requirements

ID	REQUIREMENT	IMPLEMENTATION
SAF-001	Prevent duplicate active welfare cases	`activeCasePerIts.js` blocks creation of a second active case for the same ITS number; HTTP 409 returned
SAF-002	Workflow prerequisite enforcement	Counseling form `is_complete` flag must equal 1 before workflow can advance past Counseling stage
SAF-003	Rework comment required	Reject/rework workflow actions require a mandatory comment before the transition is accepted
SAF-004	Deletion protection for referenced master data	Deletion of Jamiat, Jamaat, case types, and workflow stages blocked if active cases or users reference them
SAF-005	Audit trail immutability	`status_history` and `case_identification_income_logs` are insert-only; no UPDATE or DELETE operations permitted
SAF-006	Financial data precision	All monetary fields use `DECIMAL(15,2)` ; no floating-point types used for financial calculations
SAF-007	DB transactions for multi-table writes	InnoDB transactions wrap operations that touch multiple tables (case creation, form section saves)

5.3 Security Requirements

ID	REQUIREMENT	IMPLEMENTATION
SEC-001	Authentication tokens	JWT signed with `JWT_SECRET` (minimum 32 chars); expires in 24 hours; verified on every protected request
SEC-002	Password hashing	bcryptjs with minimum 10 rounds; plain-text passwords never stored or logged
SEC-003	Active status enforcement	`users.is_active` checked on every authenticated request; deactivated users immediately lose access without token invalidation
SEC-004	SQL injection prevention	All DB queries use `mysql2` parameterized statements; no string concatenation in SQL

ID	REQUIREMENT	IMPLEMENTATION
SEC-005	Input validation	Joi schema validation on all request bodies; request rejected with HTTP 400 and field-level errors before any DB access
SEC-006	HTTP security headers	Helmet.js sets Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security on all responses
SEC-007	CORS restriction	Only `FRONTEND_URL` env-configured origin accepted; credentials: true; all other origins rejected
SEC-008	Login rate limiting	`express-rate-limit` on `POST /api/auth/login`; configurable via env; optional IP whitelist for monitoring
SEC-009	Organizational scope isolation	Jamiat/Jamaat IDs from `user_roles` injected as SQL `WHERE IN` clauses for all non-Admin/Super Admin queries
SEC-010	File upload validation	Multer `fileFilter` validates MIME types; extension spoofing rejected; max 10 MB per file; UUID-based filenames prevent path traversal
SEC-011	RBAC per route	Every route uses `permissionMiddleware` enforcing `resource:action` against user's compiled permission set
SEC-012	Secrets management	All credentials in `.env`; `.gitignore` excludes `.env`; no hardcoded credentials in source code
SEC-013	Error masking in production	Stack traces suppressed in production; generic error message returned to client; full error logged server-side
SEC-014	ITS API credential masking	External ITS API token proxied through backend; never exposed to frontend client

5.4 Software Quality Attributes

5.4.1 Availability

TARGET	99.5% UPTIME EXCLUDING PLANNED MAINTENANCE WINDOWS
Recovery	PM2 auto-restart on process crash; systemd fallback; workflow state persisted in DB at each stage transition
Monitoring	PM2 process monitor; application error logging to console/file; Nginx access logs

5.4.2 Reliability

ATTRIBUTE	IMPLEMENTATION
Data integrity	InnoDB foreign key constraints; parameterized queries; DECIMAL types for all financial values
Graceful degradation	ITS API failure falls back to manual entry; all ITS-prefilled fields remain editable
Idempotent operations	Upsert patterns used for counseling form sections; duplicate case prevention via application-level check
Audit completeness	Every workflow transition recorded in `status_history`; income changes logged in `case_identification_income_logs`

5.4.3 Usability

ATTRIBUTE	IMPLEMENTATION
Role-appropriate UI	Sidebar navigation and page actions rendered conditionally based on user's permission set; irrelevant functions hidden
Form feedback	React Hook Form real-time validation errors; toast notifications for all async action results
Loading states	Spinner indicators for all async data fetches; optimistic updates via React Query reduce perceived latency

ATTRIBUTE	IMPLEMENTATION
Responsive design	Tailwind CSS responsive grid; functional at 768px+ viewport widths; sidebar collapses on mobile
ITS auto-fill	Applicant demographic form fields auto-populated from ITS lookup; reduces manual transcription errors

5.4.4 Scalability

ID	REQUIREMENT	IMPLEMENTATION
SC-001	Database indexing	Indexes on all FK columns, `its_number`, `case_number`, `status`, `current_workflow_stage_id`
SC-002	Pagination on all lists	All list endpoints support configurable `limit` (10–500); default 10–20; no unbounded queries
SC-003	Process management	PM2 cluster mode for multi-core utilization; systemd for OS-level process supervision
SC-004	File storage path	Local filesystem with structured `/uploads/` hierarchy; S3 migration path available with minimal code changes

5.4.5 Maintainability

ATTRIBUTE	IMPLEMENTATION
Code organization	25 backend route files (one per domain); services and utils layers separated; 31 React page components
Database migrations	Individual SQL migration files in `database/migrations/`; consolidated production dump maintained separately
Configuration externalized	All environment-specific values in `.env`; workflow stages, roles, and master data fully configurable at runtime
Audit columns	`created_at`, `updated_at`, `created_by`, `updated_by` on all core tables
Reusable UI components	16-component UI library in `frontend/src/components/ui/` ensures consistent styling and behavior system-wide

6. Database Specification

6.1 Database Technology

ATTRIBUTE	SPECIFICATION
RDBMS	MySQL 8.0+ (InnoDB storage engine)
Database Name	`baaseteen_cms`
Character Set	utf8mb4
Collation	utf8mb4_unicode_ci
Connection Pool	10 persistent connections via mysql2
Naming Convention	snake_case for all tables and columns
Financial Precision	DECIMAL(15,2) for all monetary fields
Table Count	50+ tables organized across 6 functional groups

6.2 Schema Overview

```
CORE / ACCESS
_____________
users
roles
user_roles
role_permissions
executive_levels
jamiat
jamaat

SCREENING
__________
case_identifications
case_identification_
  income_logs

REPORTING
_________
cover_letter_forms
cover_letters
fund_utilization_reports
fund_utilization_photos
fund_utilization_files
fund_utilization_sections
fund_utilization_questions

CASE WORKFLOW
_____________
cases
applicants
case_types
statuses
workflow_stages
workflow_stage_roles
workflow_stage_users
workflow_comments
status_history
case_comments
case_attachments
notifications

FINANCIAL
_________
payment_schedules
payment_schedule_
  repayments
business_assets

CHECKLIST & CLOSURE
___________________
welfare_checklist_categories
welfare_checklist_items
welfare_checklist_responses
case_closures
case_closure_documents

COUNSELING FORMS
________________
counseling_forms
personal_details
family_details
family_members
assessment
assessment_products_services
financial_assistance
financial_assistance_timeline
financial_assistance_*
economic_growth
economic_growth_projections
declaration
attachments
counseling_form_stage_permissions

MASTER DATA
___________
relations
education_levels
occupations
system_settings
```

6.3 Core Table Definitions

6.3.1 users

```
CREATE TABLE users (
    id              INT AUTO_INCREMENT PRIMARY KEY,
    username        VARCHAR(100) NOT NULL UNIQUE,
    email           VARCHAR(255) NOT NULL UNIQUE,
    password_hash   VARCHAR(255) NOT NULL,           -- bcryptjs, 10 rounds minimum
    full_name       VARCHAR(255) NOT NULL,
    role            VARCHAR(50) NOT NULL,
    is_active       TINYINT(1) DEFAULT 1,
    executive_level INT DEFAULT NULL,               -- FK to executive_levels
    jamiat_id       INT DEFAULT NULL,               -- FK to jamiat (legacy scope)
    jamaat_id       INT DEFAULT NULL,               -- FK to jamaat (legacy scope)
    photo           LONGTEXT DEFAULT NULL,          -- base64 encoded
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6.3.2 user_roles (RBAC scope assignments)

```
CREATE TABLE user_roles (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    user_id     INT NOT NULL,               -- FK (users)
    role_id     INT NOT NULL,               -- FK (roles)
    jamiat_ids  LONGTEXT DEFAULT NULL,      -- JSON array of scoped Jamiat IDs
    jamaat_ids  LONGTEXT DEFAULT NULL,      -- JSON array of scoped Jamaat IDs
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6.3.3 cases

```
CREATE TABLE cases (
    id                          INT AUTO_INCREMENT PRIMARY KEY,
    case_number                 VARCHAR(50) UNIQUE,     -- e.g. BS-2024-001
    applicant_id                INT NOT NULL,           -- FK (applicants)
    case_type_id                INT NOT NULL,           -- FK (case_types)
    status_id                   INT DEFAULT NULL,       -- FK (statuses)
    current_workflow_stage_id   INT DEFAULT NULL,       -- FK (workflow_stages)
    assigned_counselor_id       INT DEFAULT NULL,       -- FK (users)
    assigned_dcm_id             INT DEFAULT NULL,       -- FK (users)
    is_duplicate                TINYINT(1) DEFAULT 0,
    sla_status                  ENUM('on_time','at_risk','breached') DEFAULT 'on_time',
    current_stage_entered_at    DATETIME DEFAULT NULL,
    workflow_history            LONGTEXT DEFAULT NULL,  -- JSON audit array
    jamiat_id                   INT DEFAULT NULL,       -- FK (jamiat) for scoping
    jamaat_id                   INT DEFAULT NULL,       -- FK (jamaat) for scoping
    created_by                  INT DEFAULT NULL,
    created_at                  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at                  TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (applicant_id) REFERENCES applicants(id),
    FOREIGN KEY (case_type_id) REFERENCES case_types(id),
    FOREIGN KEY (current_workflow_stage_id) REFERENCES workflow_stages(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6.3.4 applicants

```
CREATE TABLE applicants (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    its_number  VARCHAR(8) UNIQUE NOT NULL,        -- 8-digit ITS ID
    full_name   VARCHAR(250) NOT NULL,
    gender      ENUM('male','female') DEFAULT NULL,
    age         INT DEFAULT NULL,
    phone       VARCHAR(50) DEFAULT NULL,
    email       VARCHAR(255) DEFAULT NULL,
    address     TEXT DEFAULT NULL,
    photo       LONGTEXT DEFAULT NULL,             -- base64 encoded
    jamiat_id   INT DEFAULT NULL,                  -- FK (jamiat)
    jamaat_id   INT DEFAULT NULL,                  -- FK (jamaat)
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6.3.5 case_identifications (Eligibility Screening)

```
CREATE TABLE case_identifications (
    id                      INT AUTO_INCREMENT PRIMARY KEY,
    its_number              VARCHAR(8) NOT NULL,
    applicant_id            INT DEFAULT NULL,       -- FK (applicants), set after ITS lookup
    eligible_in             INT DEFAULT NULL,       -- FK (case_types)
    total_family_members    INT DEFAULT NULL,
    earning_family_members  INT DEFAULT NULL,
    individual_income       DECIMAL(15,2) DEFAULT NULL,
    family_income           DECIMAL(15,2) DEFAULT NULL,
    status                  ENUM('pending','eligible','ineligible') DEFAULT 'pending',
    case_id                 INT DEFAULT NULL,       -- FK (cases), set when eligible
    reviewed_by             INT DEFAULT NULL,
    reviewed_at             DATETIME DEFAULT NULL,
    jamiat_id               INT DEFAULT NULL,
    jamaat_id               INT DEFAULT NULL,
    created_by              INT DEFAULT NULL,
    created_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6.3.6 workflow_stages

```
CREATE TABLE workflow_stages (
    id              INT AUTO_INCREMENT PRIMARY KEY,
    name            VARCHAR(100) NOT NULL,
    description     TEXT DEFAULT NULL,
    case_type_id    INT NOT NULL,               -- FK (case_types)
    sort_order      INT NOT NULL,               -- sequence position
    role_name       VARCHAR(100) DEFAULT NULL,  -- acting role string
    sla_hours       INT DEFAULT 48,
    sla_value       INT DEFAULT NULL,
    sla_unit        VARCHAR(20) DEFAULT 'hours',
    sla_warning_value INT DEFAULT NULL,
    sla_warning_unit  VARCHAR(20) DEFAULT 'hours',
    hex_color       VARCHAR(7) DEFAULT '#808080',
    is_active       TINYINT(1) DEFAULT 1,
    next_stage_id   INT DEFAULT NULL,           -- self-ref FK for linked-list chain
    FOREIGN KEY (case_type_id) REFERENCES case_types(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6.3.7 counseling_forms (Junction Record)

```
CREATE TABLE counseling_forms (
    id                      INT AUTO_INCREMENT PRIMARY KEY,
    case_id                 INT NOT NULL UNIQUE,    -- FK (cases)
    is_complete             TINYINT(1) DEFAULT 0,
    personal_details_id     INT DEFAULT NULL,       -- FK (personal_details)
    family_details_id       INT DEFAULT NULL,       -- FK (family_details)
    assessment_id           INT DEFAULT NULL,       -- FK (assessment)
    financial_assistance_id INT DEFAULT NULL,       -- FK (financial_assistance)
    economic_growth_id      INT DEFAULT NULL,       -- FK (economic_growth)
    declaration_id          INT DEFAULT NULL,       -- FK (declaration)
    attachments_id          INT DEFAULT NULL,       -- FK (attachments)
    manzoori_id             INT DEFAULT NULL,       -- FK (manzoori section)
    created_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at              TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (case_id) REFERENCES cases(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6.3.8 financial_assistance (Section 4)

```
CREATE TABLE financial_assistance (
    id                  INT AUTO_INCREMENT PRIMARY KEY,
    case_id             INT NOT NULL,           -- FK (cases)
    self_funding        DECIMAL(15,2) DEFAULT 0.00,
    rahen_available     DECIMAL(15,2) DEFAULT 0.00,
    requested_enayat    DECIMAL(15,2) DEFAULT 0.00,
    requested_qardan    DECIMAL(15,2) DEFAULT 0.00,
    approved_enayat     DECIMAL(15,2) DEFAULT NULL,
    approved_qardan     DECIMAL(15,2) DEFAULT NULL,
    total_project_cost  DECIMAL(15,2) DEFAULT 0.00,
    qh_fields           JSON DEFAULT NULL,      -- repayment schedule metadata
    created_at          TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at          TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    FOREIGN KEY (case_id) REFERENCES cases(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6.3.9 payment_schedules

```
CREATE TABLE payment_schedules (
    id                  INT AUTO_INCREMENT PRIMARY KEY,
    case_id             INT NOT NULL,
    payment_type        ENUM('qardan','enayat') NOT NULL,
    year                INT NOT NULL,           -- disbursement year
    amount_due          DECIMAL(15,2) NOT NULL,
    is_paid             TINYINT(1) DEFAULT 0,
    paid_date           DATE DEFAULT NULL,
    confirmed_by        INT DEFAULT NULL,       -- FK (users)
    created_at          TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (case_id) REFERENCES cases(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6.3.10 notifications

```
CREATE TABLE notifications (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    user_id     INT NOT NULL,                   -- FK (users) recipient
    case_id     INT DEFAULT NULL,               -- FK (cases)
    title       VARCHAR(255) NOT NULL,
    message     TEXT DEFAULT NULL,
    type        VARCHAR(50) DEFAULT NULL,       -- info / warning / error / sla_breach
    is_read     TINYINT(1) DEFAULT 0,
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (case_id) REFERENCES cases(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

6.4 Relationships and Constraints

PARENT TABLE	CHILD TABLE	TYPE	FK COLUMN	ON DELETE
applicants	cases	1:N	applicant_id	RESTRICT
case_types	cases	1:N	case_type_id	RESTRICT
cases	counseling_forms	1:1	case_id	CASCADE
cases	status_history	1:N	case_id	CASCADE
cases	case_comments	1:N	case_id	CASCADE
cases	case_attachments	1:N	case_id	CASCADE
cases	payment_schedules	1:N	case_id	CASCADE
cases	fund_utilization_reports	1:N	case_id	CASCADE
cases	welfare_checklist_responses	1:N	case_id	CASCADE
cases	cover_letter_forms	1:1	case_id	CASCADE
cases	notifications	1:N	case_id	SET NULL
users	user_roles	1:N	user_id	CASCADE
roles	role_permissions	1:N	role_id	CASCADE
workflow_stages	workflow_stage_roles	1:N	stage_id	CASCADE
family_details	family_members	1:N	family_details_id	CASCADE
financial_assistance	financial_assistance_timeline	1:N	financial_assistance_id	CASCADE
economic_growth	economic_growth_projections	1:N	economic_growth_id	CASCADE
fund_utilization_reports	fund_utilization_photos	1:N	report_id	CASCADE
jamiat	jamaat	1:N	jamiat_id	RESTRICT
welfare_checklist_categories	welfare_checklist_items	1:N	category_id	CASCADE

6.5 Data Dictionary — Key Enumerations

TABLE.COLUMN	VALUES	DESCRIPTION
cases.sla_status	on_time, at_risk, breached	Current SLA health relative to stage SLA threshold
case_identifications.status	pending, eligible, ineligible	Eligibility screening decision state
applicants.gender	male, female	Applicant gender classification
payment_schedules.payment_type	qardan, enayat	Financial assistance disbursement category
fund_utilization_reports.quarter	q1, q2, q3, q4	Quarterly reporting period
notifications.type	info, warning, error, sla_breach, assignment	Notification severity and category
counseling_forms.is_complete	0 (incomplete), 1 (complete)	All 8 section FK pointers must be non-null for is_complete = 1
users.is_active	0 (deactivated), 1 (active)	Deactivated users cannot authenticate even with valid JWT
welfare_checklist_responses.response	Y, N, NA	Per-item checklist response (Yes/No/Not Applicable)

7. System Architecture

7.1 Architectural Pattern

FACTS follows a **decoupled three-tier architecture** that cleanly separates client presentation, business logic, and data persistence into independently maintainable layers:

- **Presentation Tier** — React 18.2 Single-Page Application compiled with Vite, served as static build artifacts by Nginx. All UI rendering is client-side; server only provides JSON data.
- **Application Tier** — Node.js v16+ with Express.js handling RESTful API requests, business rule enforcement, workflow progression, PDF generation, and external service integration.
- **Data Tier** — MySQL 8.0 with InnoDB for relational transactional storage, plus local filesystem for binary file attachments and generated PDFs.

Architectural advantages: Clean API contract between frontend and backend; frontend and backend can be deployed and scaled independently; technology boundaries are explicit; testability is high due to stateless API design.

7.2 Component Diagram

```
┌──────────────────────────────────────────────────────────────────────┐
│                      CLIENT TIER (React 18 SPA)                      │
│                                                                      │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────────┐   │
│  │ Auth Context │  │ React Query  │  │  React Router v6         │ │
│  │ (JWT store)  │  │ (Server State│  │  (Protected Routes)      │ │
│  └──────────────┘  └──────────────┘  └──────────────────────────┘   │
│                                                                      │
│  Pages (31):  Login · Dashboard · Cases · CaseDetails ·              │
│               CaseIdentification · Applicants · Users ·              │
│               RoleManagement · JamiatMaster · WorkflowStages ·       │
│               WelfareChecklist · FundUtilization · Reports ·         │
│               PaymentSchedule · Notifications · Profile              │
│                                                                      │
│  Components: CounselingForm (8 sections) · CoverLetterForm ·         │
│              PaymentSchedule · FundUtilization · SignaturePad ·      │
│              FileUpload · CaseSLAStatus · WorkflowComments           │
│                                                                      │
│  UI Library (16): Button · Card · Modal · Table · Alert · Badge      │
│                   Input · Select · SearchableSelect · MultiSelect  |
│                   Pagination · Toast · Tabs · Spinner · Progress     │
└──────────────────────────────┬───────────────────────────────────────┘
                               │ HTTPS / JSON (Axios) + JWT Bearer
                               ▼
┌──────────────────────────────────────────────────────────────────────┐
│                    API TIER (Node.js + Express.js)                   │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────┐        │
│  │  Middleware Stack (executed in order per request)        │        │
│  │  Helmet (security headers)                               │        │
│  │  → CORS (FRONTEND_URL whitelist)                         │        │
│  │  → Rate Limiter (express-rate-limit)                     │        │
│  │  → JWT Auth Middleware (verify token, check is_active)   │        │
│  │  → RBAC Permission Middleware (resource:action check +  |        |
│  │      Jamiat/Jamaat scope injection)                      │        │
│  │  → Joi Validation (request body/params schema check)     │        │
│  └──────────────────────────────────────────────────────────┘        │
│                                                                      │
│  Route Modules (25):                                                 │
│  auth · users · cases · applicants · caseIdentifications             │
│  counselingForms · coverLetters · coverLetterForms                   │
│  fundUtilization · fundUtilizationMaster · quarterlyReportPdf        │
│  dashboard · notifications · attachments · roles · permissions       │
│  workflowStages · welfareChecklist · jamiat · jamaat                 │
│  caseTypes · relations · educationLevels · occupations               │
│  executiveLevels · businessAssets                                    │
│                                                                      │
│  Services: emailService (Nodemailer) · notificationService           │
│  Utils:    slaCalculator · permissionUtils · roleUtils               │
│            activeCasePerIts · quarterlyReportPdfData/Kit             │
│  PDF:      PDFKit (cover letters) · Puppeteer (quarterly reports)    │
│  Files:    Multer (upload handling, 10 MB limit, UUID naming)        │
└──────────────────────────────┬───────────────────────────────────────┘
                               │ mysql2 parameterized queries
                               │ Connection pool: 10 persistent
                               ▼
┌──────────────────────────────────────────────────────────────────────┐
│                              DATA TIER                               │
│                                                                      │
│  ┌──────────────────────────┐  ┌──────────────────────────────┐     │
│  │  MySQL 8.0 (InnoDB)      │  │  Local Filesystem            │     │
│  │  DB: baaseteen_cms       │  │  /uploads/  (Multer)         │     │
│  │  50+ tables              │  │  PDFs · images · docs        │     │
│  │  Connection pool: 10     │  │  Max 10 MB per file          │     │
│  └──────────────────────────┘  └──────────────────────────────┘     │
│                                                                      │
│  ┌──────────────────────────┐                                        │
│  │  External ITS API        │                                        │
```

```
│  │  (applicant data fetch)   │                                  │
│  │  GET /fetch-from-api/:its │                                  │
│  └───────────────────────────┘                                  │
└─────────────────────────────────────────────────────────────────┘
```

7.3 Request Lifecycle

```
Step 1: React SPA makes Axios HTTP request with Authorization: Bearer <JWT>
        |
Step 2: Express receives request
        → Helmet adds security headers (CSP, X-Frame-Options, HSTS)
        → CORS validates origin against FRONTEND_URL env var
        → Rate limiter checks request frequency (login: 5/15min; general: configurable)
        |
Step 3: JWT Auth Middleware:
        - Decodes and verifies JWT signature using JWT_SECRET
        - Re-queries DB to confirm user.is_active = 1
        - Attaches user object to req.user
        - Returns 401 if token invalid, expired, or user deactivated
        |
Step 4: RBAC Permission Middleware:
        - Checks req.user.role against required resource:action permission
        - For scoped roles (DCM, Counselor): injects WHERE jamiat_id / jamaat_id
          constraints into query context — non-admin users cannot bypass
        - Returns 403 if insufficient permission
        |
Step 5: Joi Validation:
        - Validates request body and query params against defined schema
        - Returns 400 with field-level error details on validation failure
        |
Step 6: Route Handler (Business Logic):
        - Executes parameterized MySQL queries via connection pool
        - Calls services: emailService, notificationService, slaCalculator
        - Invokes PDF generators (PDFKit / Puppeteer) as needed
        - Handles file I/O via Multer streams
        |
Step 7: Response:
        - Returns JSON payload with standard HTTP status code
        - React Query caches response; UI state updates reactively
        - Toast notification displayed to user
```

7.4 Directory Structure

```
facts-cms/
├── backend/
│   ├── index.js                        # Express server entry; mounts all routers
│   ├── config/
│   │   └── database.js                 # mysql2 connection pool (10 connections)
│   ├── middleware/
│   │   ├── auth.js                     # JWT verify, is_active check, req.user attach
│   │   └── permissionMiddleware.js     # RBAC resource:action + scope injection
│   ├── routes/                         # 25 route files (one per module domain)
│   │   ├── auth.js                     # Login, profile, change-password, logout
│   │   ├── users.js                    # User CRUD, toggle-active, Excel import
│   │   ├── cases.js                    # Case CRUD, workflow actions, comments
│   │   ├── applicants.js               # Applicant CRUD, ITS lookup, bulk import
│   │   ├── caseIdentifications.js      # Eligibility screening endpoints
│   │   ├── counselingForms.js          # 8-section form get/update/complete
│   │   ├── coverLetters.js             # PDF generation and download
│   │   ├── coverLetterForms.js         # Cover letter form data
│   │   ├── fundUtilization.js          # Quarterly reports, photos, files
│   │   ├── fundUtilizationMaster.js    # Form builder sections/questions
│   │   ├── quarterlyReportPdf.js       # Quarterly PDF generation
│   │   ├── dashboard.js                # Overview, activities, pipeline
│   │   ├── notifications.js            # Notification CRUD + SLA trigger
│   │   ├── attachments.js              # File upload/download/delete
│   │   ├── roles.js                    # Role CRUD, permissions, stats
│   │   ├── permissions.js              # Permission checks and summaries
│   │   ├── workflowStages.js           # Stage CRUD, reorder, role mapping
│   │   ├── welfareChecklist.js         # Checklist categories/items/responses
│   │   ├── jamiat.js / jamaat.js       # Org hierarchy CRUD and import
│   │   └── [caseTypes, relations, etc.]  # Master data tables (5 modules)
│   ├── services/
│   │   ├── emailService.js             # Nodemailer SMTP with HTML templates
│   │   └── notificationService.js      # In-app notification creation
│   ├── utils/
│   │   ├── slaCalculator.js            # SLA status: on_time/at_risk/breached
│   │   ├── activeCasePerIts.js         # One-active-case-per-ITS enforcement
│   │   ├── permissionUtils.js          # Reusable permission check helpers
│   │   ├── roleUtils.js                # Role resolution and hierarchy logic
│   │   ├── quarterlyReportPdfData.js   # Data aggregation for quarterly PDF
│   │   └── quarterlyReportPdfKit.js    # PDFKit quarterly report builder
│   ├── templates/
│   │   └── quarterlyReportPdfTemplate.js # HTML template for Puppeteer PDF
│   └── uploads/                        # Runtime file storage (gitignored)
│       ├── cases/                      # Case attachments and closure docs
│       ├── fund-utilization/           # Photos and files for utilization reports
│       └── cover-letters/              # Generated cover letter PDFs
├── frontend/
│   ├── src/
│   │   ├── App.js                      # React Router v6 with protected routes
│   │   ├── contexts/
│   │   │   └── AuthContext.js          # JWT token, user object, permissions
│   │   ├── pages/                      # 31 page components
│   │   ├── components/
│   │   │   ├── ui/                     # 16 shared UI components
│   │   │   ├── CounselingForm.js       # 8-section tabbed counseling wizard
│   │   │   ├── CoverLetterForm.js      # Cover letter with pre-populated data
│   │   │   ├── PaymentSchedule.js      # Payment matrix and disbursement
│   │   │   ├── SignaturePad.js         # Digital signature (draw or file mode)
│   │   │   └── CaseSLAStatus.js        # SLA badge and countdown display
│   │   └── utils/
│   │       ├── permissionUtils.js      # usePermission hook
│   │       └── generateCoverLetterPDF.js # Client-side cover letter helper
│   └── build/                          # Production build — served by Nginx
├── database/
│   ├── baaseteen_production_database.sql # Consolidated full schema
│   └── migrations/                     # Individual schema migration files
├── .env                                # Environment variables (gitignored)
├── package.json                        # Root scripts (dev, frontend, backend)
└── .gitignore
```

8. Data Flow Specifications

8.1 Context Diagram (Level 0)

```
                        ┌──────────────────────────┐
                        │                          │
  ITS Number ──────────►│                          │──── Applicant Profile
  Credentials ─────────►│                          │
  Case Data ───────────►│                          │──── Case Status / History
  Form Sections ───────►│          FACTS           │
  Workflow Actions ────►│    Case Management       │──── Cover Letter PDF
  Disbursement Confirm ►│        System            │
  Fund Utilization ────►│                          │──── Quarterly Report PDF
  File Uploads ────────►│                          │
                        │                          │──── Excel Exports
  External ITS API ────►│                          │
  SMTP Server ─────────►│                          │──── Email Notifications
                        └──────────────────────────┘
```

8.2 Case Lifecycle Data Flow

```
Field Worker / DCM              System                      Approver / Executive
       │                          │                                 │
       ├── Create Case ID ───────►│                                 │
       │   (ITS lookup → auto-fill)│                                │
       │                          ├── Query ITS API (if no local)  │
       │                          ├── Duplicate ITS check          │
       │◄── Case ID record saved ─┤                                 │
       │                          │                                 │
       │◄──────── Reviewer marks Eligible / Ineligible ────────────►│
       │                          ├── If Eligible: Auto-create Case│
       │                          ├── Assign DCM + Counselor       │
       │◄── Case created (DRAFT) ─┤                                 │
       │                          │                                 │
       ├── Fill Counseling Form ──►│ (8 sections; save per section) │
       │                          ├── Track section completion     │
       │◄── Form progress saved ──┤                                 │
       │                          │                                 │
       ├── Submit for Welfare ────►│                                │
       │                          ├── Validate is_complete = 1     │
       │                          ├── Advance stage pointer        │
       │                          ├── Record status_history        │
       │                          ├── Notify Welfare Reviewer ─────►│
       │                          │◄── Welfare Approve/Rework ──────┤
       │                          ├── If Rework: reset SLA timer   │
       │                          ├── If Approved: Advance to ZI   │
       │                          │◄── ZI Approve / Reject ────────┤
       │                          ├── Advance through KG / Ops Lead│
       │                          ├── Generate Cover Letter PDF    │
       │                          ├── Executive Approval (1–5 lvls)│
       │                          ├── Manzoori (Sanction)          │
       │                          ├── Create Payment Schedule      │
       │                          ├── Finance Confirms Disbursements│
       │                          ├── Quarterly Fund Utilization    │
       │                          └── Case Closure                 │
```

8.3 SLA Calculation Flow

```
Case advances to new workflow stage
        │
        ├── Record current_stage_entered_at = NOW()
        │   Reset sla_status = "on_time"
        │
SLA check triggered (POST /api/notifications/check-sla or scheduled event)
        │
        ├── For each active case (non-terminal status):
        │       elapsed = NOW() - current_stage_entered_at
        │       stage.sla_value + stage.sla_unit → convert to hours
        │       stage.sla_warning_value + sla_warning_unit → convert
        │
        ├── If elapsed ≥ sla_value (breach):
        │       Set sla_status = "breached"
        │       Record sla_breached_at timestamp
        │       Notify: assigned_counselor + relevant approver + admin
        │       Notification type: "error"
        │
        ├── If elapsed ≥ sla_warning_value but < sla_value (at risk):
        │       Set sla_status = "at_risk"
        │       Notify: assigned_counselor + relevant approver
        │       Notification type: "warning"
        │
        └── Else:
                sla_status = "on_time"  (returned on case list API responses)
```

8.4 ITS Auto-Fetch Data Flow

```
User types ITS number in Case Identification / Applicant form
        │
        ├── Frontend: GET /api/applicants/fetch-from-api/:itsNumber
        │
        ├── Backend: Query local applicants table by its_number
        │       If found → return record with source: "database"
        │       (No external call needed — cached locally)
        │
        ├── If not found locally → proxy to external ITS API endpoint
        │       Timeout limit: 5 seconds (graceful fallback on failure)
        │       Parse JSON response → map to applicant schema fields:
        │         full_name, age, gender, phone, email, photo (base64),
        │         jamiat_name, jamaat_name, address
        │       Return mapped object with source: "api"
        │
        ├── Frontend auto-populates form fields from returned object
        │       Editable fields remain open for user corrections
        │
        └── User reviews → saves applicant record to local DB
```

8.5 Module Dependency Map

```
┌──────────────────────────────────────────────────────────────┐
│              Authentication & RBAC Module                    │
│      JWT + user_roles + role_permissions                     │
│      (Required by ALL modules — every route is guarded)      │
└──────────────────────────┬───────────────────────────────────┘
                           │
┌──────────────────────────┴───────────────────────────────────┐
│              User Management & Role Management               │
│      (Required for counselor/DCM/approver assignment)        │
└──────┬──────────────────┬───────────────────────────────────┘
       │                  │
┌──────┴──────────┐  ┌────┴──────────────────────────────────┐
│  Master Data    │  │         Case Identification Module    │
│  Jamiat/Jamaat  │  │  (Eligibility screening — entry point) │
│  CaseTypes      │  └──────────────────┬────────────────────┘
│  WorkflowStages │                     │ (eligible → auto-create)
│  Relations      │                     ▼
│  EducationLvls  │  ┌──────────────────────────────────────┐
│  Occupations    │  │           Case Management Module     │
│  ExecutiveLvls  │  │  (core — all other modules depend on it) │
└─────────────────┘  └──┬──┬──┬──┬──┬──┬──┬──┬──┬──────────┘
                        │  │  │  │  │  │  │  │  │
      ┌─────────────────┘  │  │  │  │  │  │  │  └─────────────────┐
      ▼                    │  │  │  │  │  │  │                    ▼
Counseling Forms        Payment │  │  │  │  │  Welfare       Notifications
(8-section form)       Schedule │  │  │  │  │  Checklist     (cross-cutting)
                                │  │  │  │  │
                          Cover │  │  │  Fund Utilization
                          Letter │  │  │  (quarterly reports)
                                │  │  │
                        Attach- │  Dashboard & Analytics
                        ments   │  (reads from all modules)
                                │
                            Audit Trail
                            (status_history, workflow_comments,
                             income_logs — immutable writes)

Cross-cutting: SLA Calculator  (reads workflow_stages → writes sla_status)
Cross-cutting: Notification Service (triggered by Case, Workflow, SLA events)
Cross-cutting: Email Service (optional SMTP layer over Notification Service)
```

9. Deployment Specification

9.1 Production Environment

COMPONENT	SPECIFICATION
Operating System	Linux (Ubuntu 20.04+ recommended)
Web Server / Proxy	Nginx with SSL termination (Let's Encrypt or commercial cert); HTTP → HTTPS redirect enforced
Backend Runtime	Node.js v16+ managed by PM2 (cluster mode) or systemd for auto-restart on crash
Backend Port	5000 (internal only; Nginx proxies `/api` and `/uploads` to this port)
Frontend Delivery	React production build (`npm run build`) served as static files by Nginx with SPA fallback (`try_files $uri /index.html`)
Database	MySQL 8.0, database name: `baaseteen_cms` , bound to localhost (no external port exposure)
File Storage	Local filesystem at `./uploads/` ; directories: `cases/` , `fund-utilization/` , `cover-letters/`
Max Upload Size	10 MB per file (`client_max_body_size 10M` in Nginx; `MAX_FILE_SIZE=10485760` in .env)
Firewall	Only ports 22 (SSH), 80 (HTTP redirect), 443 (HTTPS) open externally

9.2 Nginx Configuration Reference

```
server {
    listen 443 ssl;
    server_name your-domain.com;

    # React SPA — serve build artifacts
    location / {
        root /path/to/frontend/build;
        try_files $uri /index.html;
    }

    # Express API — proxy to Node.js
    location /api {
        proxy_pass http://localhost:5000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }

    # Uploaded files — proxy through Node.js for auth-gated access
    location /uploads {
        proxy_pass http://localhost:5000/uploads;
    }

    client_max_body_size 10M;
}
```

9.3 Deployment Steps

```
# 1. Build React frontend
cd frontend && npm install && npm run build

# 2. Configure production .env (see Appendix E for all variables)
#    NODE_ENV=production
#    JWT_SECRET=<cryptographically-random-32+-char-string>
#    DB credentials pointing to production MySQL instance

# 3. Import database schema
mysql -u root -p -e "CREATE DATABASE baaseteen_cms;"
mysql -u root -p baaseteen_cms < database/baaseteen_production_database.sql

# 4. Install backend dependencies and start with PM2
cd backend && npm install
pm2 start index.js --name facts-backend
pm2 save && pm2 startup

# 5. Configure Nginx virtual host (see 9.2 above)
sudo nginx -t && sudo systemctl reload nginx

# 6. Post-deployment checklist
#    ✓ Default admin password changed immediately (admin / password → new password)
#    ✓ JWT_SECRET set to random 32+ character string
#    ✓ uploads/ directory has write permissions for node process
#    ✓ Database backups scheduled (daily recommended via mysqldump cron)
#    ✓ Firewall configured: only ports 80, 443, 22 open
#    ✓ SSL certificate installed and auto-renewal configured
```

9.4 Local Development Environment

COMPONENT	SPECIFICATION
Backend	`npm run backend` → nodemon watching `backend/` on port 5000
Frontend	`npm run frontend` → Vite dev server on port 3000 with HMR
Combined	`npm run dev` → concurrently starts both backend and frontend
Database	Local MySQL 8.0; configure `DB_HOST=localhost` in `.env`
API URL	`REACT_APP_API_URL=http://localhost:5000` in frontend `.env`
Default login	Username: `admin` / Password: `password` (change immediately after setup)

9.5 External Integration Summary

INTEGRATION	TECHNOLOGY	PURPOSE	FALLBACK BEHAVIOUR
ITS Institutional API	External REST / HTTPS	Auto-fetch applicant demographics and photo by ITS number	Manual data entry if API unreachable; timeout after 5 s
SMTP Email	Nodemailer v6.9	Optional email alerts for case status changes and notifications	In-app notifications still sent; email silently skipped if SMTP unavailable
PDFKit	Node.js library v0.14	Cover letter PDF generation (required)	N/A — library must be installed
Puppeteer	Headless Chrome v24	Quarterly fund utilization report PDF via HTML template	N/A — required for quarterly reports
xlsx library	Node.js library v0.18	Excel template download and bulk import (users, applicants, Jamiat)	CSV format is an acceptable manual alternative
Multer	Node.js middleware v1.4	Multipart file upload handling for attachments and photos	N/A — required for all file operations

10. Appendices

Appendix A: Use Case Catalog

UC ID	USE CASE	PRIMARY ACTOR	PRIORITY
UC-01	Login to system	All Users	Critical
UC-02	Logout from system	All Users	Critical
UC-03	View own profile	All Users	Medium
UC-04	Change own password	All Users	High
UC-05	Create user account	Admin / Super Admin	Critical
UC-06	Edit user account	Admin / Super Admin	High
UC-07	Deactivate / reactivate user	Admin / Super Admin	High
UC-08	Assign executive level to user	Admin / Super Admin	High
UC-09	Bulk import users via Excel	Admin / Super Admin	Medium
UC-10	Manage roles and permissions	Super Admin	Critical
UC-11	Assign permissions to role	Super Admin	Critical
UC-12	Create Jamiat (regional unit)	Super Admin	High
UC-13	Create Jamaat (sub-regional unit)	Super Admin	High
UC-14	Import Jamiat / Jamaat via Excel	Super Admin	Medium
UC-15	Manage case types	Super Admin	High
UC-16	Manage workflow stages	Super Admin	Critical
UC-17	Assign roles to workflow stage	Super Admin	Critical
UC-18	Reorder workflow stages	Super Admin	High
UC-19	Create case identification (screening)	DCM / Admin	Critical
UC-20	Auto-fetch applicant data via ITS	System	Critical
UC-21	Mark case identification as eligible	Welfare Reviewer / DCM	Critical
UC-22	Mark case identification as ineligible	Welfare Reviewer / DCM	High
UC-23	Create applicant record	DCM / Admin	Critical
UC-24	Edit applicant demographics	DCM / Counselor	High
UC-25	Bulk import applicants via Excel	Admin	Medium
UC-26	Create case from eligible identification	System (auto)	Critical
UC-27	Assign DCM to case	Admin / DCM	Critical
UC-28	Assign counselor to case	DCM / Admin	Critical
UC-29	View case list (scoped by role)	All authenticated roles	Critical
UC-30	View case detail	All authorized roles	Critical
UC-31	Add comment to case	DCM / Counselor / Approver	High
UC-32	Upload case attachment	DCM / Counselor	High
UC-33	Download case attachment	All authorized roles	High

UC ID	USE CASE	PRIMARY ACTOR	PRIORITY
UC-34	Fill counseling form — Personal Details	Counselor / DCM	Critical
UC-35	Fill counseling form — Family Details	Counselor / DCM	Critical
UC-36	Fill counseling form — Assessment	Counselor / DCM	Critical
UC-37	Fill counseling form — Financial Assistance	Counselor / DCM	Critical
UC-38	Fill counseling form — Economic Growth Projections	Counselor / DCM	Critical
UC-39	Fill counseling form — Declaration & Signatures	Counselor	Critical
UC-40	Mark counseling form complete	Counselor / DCM	Critical
UC-41	Submit case for Welfare Review	DCM	Critical
UC-42	Complete welfare checklist	Welfare Reviewer	Critical
UC-43	Approve case at Welfare Review stage	Welfare Reviewer	Critical
UC-44	Reject / return case for rework	Welfare Reviewer / ZI / Executive	High
UC-45	Approve case at ZI / KG / Ops Lead stages	ZI / KG / Operations Lead	Critical
UC-46	Generate cover letter PDF	System / Operations Lead	High
UC-47	Edit and finalize cover letter form	DCM / Operations Lead	High
UC-48	Approve at Executive tier (1–5 levels)	Executive (by level)	Critical
UC-49	Sanction case (Manzoori)	Admin / Authorized Executive	Critical
UC-50	View / confirm payment schedule	Finance	Critical
UC-51	Confirm disbursement of payment installment	Finance	Critical
UC-52	Create quarterly fund utilization report	Counselor / DCM	High
UC-53	Upload fund utilization photos	Counselor / DCM	High
UC-54	Generate quarterly fund utilization PDF	System / DCM	High
UC-55	Close case	DCM / Admin	High
UC-56	Upload case closure documents	DCM	High
UC-57	View dashboard statistics	All authenticated roles	High
UC-58	View case pipeline by workflow stage	Admin / DCM	High
UC-59	View recent activity feed	Admin / DCM	Medium
UC-60	View and dismiss notifications	All authenticated roles	High
UC-61	Trigger SLA check	System / Admin	Medium
UC-62	Manage welfare checklist categories and items	Super Admin	Medium
UC-63	Configure fund utilization form sections/questions	Super Admin	Medium
UC-64	Manage master data (relations, education, occupations)	Super Admin / Admin	Medium
UC-65	Manage executive levels	Super Admin	High

Appendix B: Error Handling Specification

ERROR SCENARIO	HTTP STATUS	HANDLING STRATEGY	CLIENT MESSAGE
Invalid credentials	401	Respond without revealing which field is wrong	"Invalid credentials."
Deactivated user attempting login	401	is_active check in auth middleware	"Account is deactivated. Contact administrator."
Expired / invalid JWT	401	JWT verify failure caught in auth middleware	"Session expired. Please log in again."
Insufficient permission (RBAC)	403	permissionMiddleware blocks request before handler	"You do not have permission to perform this action."
Scope violation (wrong Jamiat)	403	Scope injection prevents data from being returned	"Access to this resource is restricted."
Resource not found	404	Route handler returns 404 on empty DB result	"The requested resource was not found."
Duplicate active case for ITS	409	activeCasePerIts utility blocks creation	"An active case already exists for this ITS number."
Deletion of entity with FK dependencies	409	FK constraint check before DELETE	"Cannot delete: this record has dependent data."
Joi validation failure	400	Joi middleware returns field-level errors	Field-specific validation messages per input.
Missing required field	400	Joi schema required() rule	"[field] is required."
File upload: MIME type rejected	400	Multer fileFilter blocks disallowed types	"Invalid file type. Allowed: PDF, JPEG, PNG, DOCX."
File upload: exceeds size limit	400	Multer limits.fileSize enforced	"File size exceeds the 10 MB limit."
Workflow stage prerequisite not met	422	Business logic checks before stage advancement	"Cannot advance: counseling form is not complete."
ITS API unreachable	503 (graceful)	5-second timeout; fallback to manual entry	"ITS service unavailable. Please enter details manually."
PDF generation failure	500	Error caught; logged server-side	"PDF generation failed. Please try again."
Database connection failure	500	Pool error logged; generic response returned	"Service temporarily unavailable. Please try again."

Appendix C: File Upload Specifications

UPLOAD CONTEXT	ALLOWED MIME TYPES	MAX SIZE	MAX COUNT	STORAGE PATH
Case Attachments	PDF, JPEG, PNG, DOCX, DOC, XLSX	10 MB	10 per request	`uploads/cases/{caseId}/`
Case Closure Documents	PDF, JPEG, PNG, DOCX	10 MB	10 per case closure	`uploads/cases/{caseId}/closure/`
Fund Utilization Photos	JPEG, PNG	10 MB	Multiple per report	`uploads/fund-utilization/{reportId}/photos/`
Fund Utilization Files	PDF, JPEG, PNG, DOCX, XLSX	10 MB	Multiple per report	`uploads/fund-utilization/{reportId}/files/`

UPLOAD CONTEXT	ALLOWED MIME TYPES	MAX SIZE	MAX COUNT	STORAGE PATH
Excel Bulk Import	`application/vnd.openxmlformats-officedocument.spreadsheetml.sheet`	10 MB	1 per request	Temporary (not persisted)
Generated Cover Letter PDFs	application/pdf (system-generated)	N/A	1 per case	`uploads/cover-letters/`

File naming convention: All uploaded files stored with UUID-based names (`uuid-v4.ext`). Original filenames are sanitized and recorded in the database only. This prevents directory traversal attacks and filename collisions.

Appendix D: API Endpoint Inventory

MODULE	BASE URL	KEY OPERATIONS	COUNT
Authentication	`/api/auth`	login, register, profile, change-password, logout	6
Users	`/api/users`	CRUD, toggle-status, assign-executive-level, import-excel, fetch-from-api	16
Cases	`/api/cases`	CRUD, workflow actions (welfare/ZI/KG/executive/manzoori), comments, payment-schedule, close, closure-docs	27
Case Identifications	`/api/case-identifications`	List, get, create, review (eligible/ineligible), edit, revert-to-pending, check-its	7
Applicants	`/api/applicants`	CRUD, ITS lookup, fetch-from-api, bulk-import, bulk-fetch, import-excel, export-template	13
Counseling Forms	`/api/counseling-forms`	Get full form, update per section (8 sections), mark complete	10
Cover Letters	`/api/cover-letters`	Generate PDF, get for case, download	3
Cover Letter Forms	`/api/cover-letter-forms`	Get by case, create/update, submit	3
Dashboard	`/api/dashboard`	Overview stats, recent-activities, case-pipeline	3
Notifications	`/api/notifications`	List, mark-read, mark-all-read, unread-count, delete, check-sla trigger	6
Attachments	`/api/attachments`	Upload single/multiple, list by case, download, delete, stats	6
Roles	`/api/roles`	CRUD, available-permissions, assign, unassign, stats	10
Permissions	`/api/permissions`	Check, counseling-form-access, counseling-form-roles, admin-roles, summary	5
Workflow Stages	`/api/workflow-stages`	CRUD, restore, reorder, manage roles/users per stage, status-mappings	16
Welfare Checklist	`/api/welfare-checklist`	Category CRUD + reorder, Item CRUD + grouped view, responses, status	14
Fund Utilization	`/api/cases/:id/fund-utilization`	CRUD reports, upload photos, upload files, delete photos, delete files	9
Fund Utilization Master	`/api/fund-utilization-master`	Form-config, sections CRUD + reorder, questions CRUD + reorder	11
Quarterly Report PDF	`/api/cases/:id/quarterly-report`	Generate and download quarterly PDF	1

MODULE	BASE URL	KEY OPERATIONS	COUNT
Jamiat / Jamaat	`/api/jamiat`, `/api/jamaat`	CRUD, import, export, template download, by-jamiat filter	14
Master Data	`/api/case-types`, `/api/relations`, `/api/education-levels`, `/api/occupations`, `/api/executive-levels`	CRUD per table; executive-levels adds reorder and active-list endpoints	25+
Business Assets	`/api/business-assets`	Get by case, create/update, delete	3

Total: 150+ endpoints across 25 route files

Appendix E: Environment Variables Reference

VARIABLE	REQUIRED	DEFAULT	DESCRIPTION
`DB_HOST`	Yes	—	MySQL server hostname (e.g., `localhost`)
`DB_USER`	Yes	—	MySQL username with access to `baaseteen_cms`
`DB_PASSWORD`	Yes	—	MySQL user password
`DB_NAME`	Yes	`baaseteen_cms`	Database name
`JWT_SECRET`	Yes	—	JWT signing key — minimum 32 characters; use a cryptographically random value
`JWT_EXPIRES_IN`	Yes	`24h`	Token expiry duration (e.g., `24h`, `7d`)
`PORT`	Yes	`5000`	Express server listening port
`NODE_ENV`	Yes	—	`development` or `production`
`FRONTEND_URL`	Yes	—	React app origin for CORS allowlist (e.g., `https://your-domain.com`)
`EMAIL_HOST`	No	—	SMTP host (e.g., `smtp.gmail.com`); email features disabled if omitted
`EMAIL_PORT`	No	`587`	SMTP port for TLS
`EMAIL_USER`	No	—	SMTP authentication username / email address
`EMAIL_PASS`	No	—	SMTP password or app-specific password
`EMAIL_FROM`	No	—	From address displayed on outbound system emails
`MAX_FILE_SIZE`	No	`10485760`	Maximum upload file size in bytes (default: 10 MB)
`UPLOAD_PATH`	No	`./uploads`	Filesystem directory for file storage; must be writable by Node process
`REACT_APP_API_URL`	Yes	—	Backend API base URL consumed by frontend (e.g., `http://localhost:5000`)
`REACT_APP_ENV`	No	—	`development` or `production` — controls frontend debug behaviour
`LOGIN_RATE_LIMIT_SKIP_IPS`	No	—	Comma-separated IPs excluded from login rate limiting (e.g., internal health-check monitors)